

队伍编号	202601139
题号	A 题

长江十年禁渔下鱼类食物链恢复、动力学复杂化与入侵风险的耦合建模

摘 要

长江十年禁渔实施后，四大家鱼资源量回升，长江江豚和长江鲟种群亦呈恢复态势，但资源枯竭、江湖连通性受阻、水体污染与外来物种入侵等问题仍较突出。围绕“资源恢复能否稳定转化为生态系统健康改善”核心问题，本文分别描述流域平均食物网、珍稀物种响应和三级食物链的长期行为，再用综合情景评价比较污染与入侵压力下能够保留的禁渔收益。

问题一在流域平均尺度上配对四类基础资源与四大家鱼功能群。以 2025 年四大家鱼资源量恢复至禁渔前 1.8 倍作单参数标定，湖北江段单网次渔获量提升 120% 留作局地对照。结果显示，2025 年四大家鱼总生物量为禁渔前的 1.805 倍，单位捕捞努力量渔获量 (CPUE*) 为 1.838 倍；摄食压力指数升至 4.213，基础资源量末值降至 0.204。鱼类回升早于底层资源恢复。

问题二把上游资源-鱼群轨迹压缩为综合生态状态指数，用它调节中层猎物承载力，并设置高阻隔、无人工放流和污染情景。基线下长江江豚与长江鲟末值为 1.133 倍和 1.140 倍，高阻隔时降至 0.901 倍和 0.856 倍；取消人工放流后长江鲟降至 0.519 倍，污染情景下两者为 0.610 倍和 0.704 倍。对照结果显示，通道阻隔同时压低两类物种，放流变化的主要响应落在长江鲟。

长期行为部分采用同一 Hastings-Powell 三级食物链。参数 K 增大时，代表点 $K = 1.795$ 、 2.665 、 4.800 的峰谷差和峰间隔离散程度随之增大，轨道由规则振荡转为复杂非线性波动。短时间窗分析显示，首个连续正李雅普诺夫指数区间位于 $K = 3.2186$ 附近；75 组扩展测试中，仅 5 组在 $K = 3.20 \sim 3.22$ 区间保留由负转正的跨越。故本文仅报告“系统动力学复杂性随承载力提高而增强”的方向性结论。

综合评价取积分后期的饵料资源、长江江豚、长江鲟和入侵压力指标，并用熵权与 CRITIC 平均权重计算情景得分。三种情景依次得到 0.662、0.423 和 0.155；2000 次局部参数扰动无一改变排序，稳定率为 100%。这一顺序与“水体污染首先削弱禁渔收益，外来物种入侵进一步导致系统状态恶化”相对应。

十年禁渔后，鱼类资源已经回升，但资源恢复不会自动转化为系统整体健康改善：模型中的底层资源、珍稀物种和长期轨道变化幅度各异。若不同步推进水质治理、江湖连通性修复、珍稀物种定向放流与入侵防控，恢复收益仍可能在向高营养级传递和多重压力叠加时减弱。

关键词：长江十年禁渔；四大家鱼；长江江豚；长江鲟；三级食物链；李雅普诺夫指数

一、问题重述

长江流域是我国淡水渔业最发达、生物多样性最丰富的区域之一，但长期受过度捕捞、工业废水排放、水域塑料污染以及葛洲坝、三峡大坝等水利工程的综合影响，鱼类洄游通道被阻断，渔业资源持续衰退，典型食物链结构发生退化。自 2021 年 1 月 1 日实施长江十年禁渔政策以来，长江流域生态环境已出现显著变化：局部水域水质改善，水草等水生植被逐步恢复；2025 年监测数据显示，青、草、鲢、鳙四大家鱼资源量已恢复至禁渔前的约 1.8 倍，表明禁渔对基础资源和经济鱼类具有显著的修复作用。

但生态系统的恢复并非总是朝着良性方向发展。部分水域在禁渔后出现鱼类种群数量增长过快、过度摄食水生植物和浮游动物的现象，进而导致水体浑浊、底栖植被大面积减少，部分水域甚至形成大面积“水下荒漠”。长江江豚种群数量已回升至约 1249 头，2024 年长江鲟自然繁殖产卵场重新被发现；而鳄雀鳝、巴西龟等外来入侵物种已在鄱阳湖等流域支流定殖并维持种群。十年禁渔政策确实取得了显著的资源恢复成效，但恢复后的生态系统未必更加健康，营养级失衡、江湖连通性恢复不足以及外来物种入侵扩散等问题依然突出。因此，有必要分别度量“资源恢复”与“系统健康”，并追踪两者在各营养级上的差异^[1-2]。

题目要求依次回答以下五个子问题：

- 问题一：分析禁渔政策对水生植物、浮游动物及四大家鱼种群动态的影响。
- 问题二：研究禁渔产生的生态收益能否向上传递至长江鲟和长江江豚等顶级捕食者。
- 问题三：构建长江流域简化鱼类食物链模型并探讨其长期演化规律。
- 问题四：判别三级食物链系统在不同承载力条件下的动力学行为（稳定、周期振荡或混沌）。
- 问题五：将工业废水排放、塑料污染和外来物种入侵等胁迫因子纳入统一评价框架，综合评估十年禁渔对长江渔业生态系统的整体效应，并提出针对性的治理对策建议

各问统一采用以下时间口径：2021 年为政策启动基准年，2022 年为公报基准年，2025 年为恢复效果对照年^[3]。

二、问题分析

2.1 总体分析

四大家鱼回升、江豚回暖和长江鲟产卵场重现，说明禁渔后的恢复已经出现；“秃海”、通道受阻和入侵扩散则会削弱这部分收益。这些现象相互制约，因此分析不止于单项指标，也不把不同营养级中的某一层变化代替系统状态。

五问沿同一传递关系展开，上游对象不在后续各问重复建立。问题一先辨别恢复落在哪一层，问题二追踪这部分收益能否传到珍稀物种；问题三、四转向长期，考察系统会不会随承载条件变化而出现更复杂的轨道；问题五将污染和入侵加入比较。全文因此由“恢复是否发生”转向“恢复是否稳得住、是否算健康”。

2.2 问题一分析

问题一同时考察“鱼类资源量增加”、“鱼类增长的驱动机制”和“基础资源的承载压力变化”。若把四大家鱼与水草、浮游植物、浮游动物、底栖饵料各自合并，模型仍能匹配 2025 年 1.8 倍的总量，却无法分辨哪一层资源先承压，也难以解释“水下荒漠”等底层退化现象。因此，方程中的四类资源必须与四大家鱼的食性逐项对应。

题目给出的两类观测数据分别对应两种尺度：2025 年四大家鱼 1.8 倍恢复量对应全流域平均口径，可用于约束系统总量参数；湖北江段单网次渔获量提升 120

2.3 问题二分析

四大家鱼资源恢复并不等同于珍稀保护物种同步受益。两类物种的受限环节并不相同：长江江豚受中层猎物供给约束，长江鲟不只受食源限制，还受到洄游通道和人工增殖放流的影响。把食源供给、江湖连通性和放流补偿分别写入方程后，才能辨明种群终态差异归因于哪一环节。

问题二将 2022 年公报基准值、江湖阻隔数据及人工放流信息分别纳入模型状态量、损失项与控制量，并设置高阻隔、无人工放流和水体污染三类对照情景，从而用种群终态差异比较各压力源的作用。

2.4 问题三分析

系统长期动力学行为的识别比短期政策效应分析更依赖简洁的模型结构。若沿用前两问的高维划分，参数空间将急剧膨胀，反而难以归纳系统轨道类型。因此，我们在问题三中采用“基础资源供给-中层鱼类转移-顶级捕食者反馈”的最简三级食物链闭环，将环境承载力变化抽象为无量纲控制参数 K ，在同一参数轴上分析系统从规则有界振荡向复杂非线性波动的演化过程。

仅靠轨道形态不足以支撑系统行为分类，还需结合统一时间窗内的后期轨道统计量。通过综合分析波动幅度、峰谷差及峰间间隔的离散程度，才能为系统行为分类提供可重复验证的定量依据。

2.5 问题四分析

问题三揭示的复杂轨道只是现象，问题四旨在判别该现象是否伴随初值敏感性。此处最需避免的误区是将有限时间窗内的不规则轨道误判为稳定混沌。若正李雅普诺夫指数仅在特定积分步长、时间窗口或初始条件下出现，则其更可能是条件性数值伪影，而非系统固有的动力学阈值。

因此，除基准参数扫描外，我们在保持方程组不变的前提下，调整积分时长、步长及局部初始条件，检验正李雅普诺夫指数区间的鲁棒性。若不同数值设置下的判别结果一致，则系统动力学复杂性增强的趋势具有可靠解释力；若结果随数值设置显著波动，则仅能得出“系统复杂波动程度增强”的保守结论。

2.6 问题五分析

前四问的计算表明，禁渔的恢复收益在生态系统不同层级并非同步传递：基础食源、经济鱼类、珍稀物种及系统长期动力学行为的响应存在明显差异。问题五据此使用统一的时间窗和指标正向化规则，选取基础食源、长江江豚、长江鲟和外来入侵压力四类核心指标，比较水体污染与外来物种入侵的复合生态效应。

综合得分之外，熵权法、CRITIC 法、等权法及局部参数扰动还用于检查排序是否改变；若排序均一致，”水体污染首先削弱禁渔收益，外来物种入侵进一步恶化系统状态”的结论具有稳健性。

三、模型假设

全文共用同一时间口径、状态尺度和扰动进入方式，作如下假设。

1. 十年尺度内，长江流域宏观水环境用归一化承载力表示，不再逐一建立局地空间网格。
2. 四大家鱼按食性拆分为草鱼、鲢鱼、鳙鱼、青鱼四个功能群，分别对应水草、浮游植物、浮游动物和底栖饵料。
3. 江豚与长江鲟恢复受食源约束；长江鲟额外依赖人工放流，并对大坝阻隔更敏感。
4. 污染通过降低承载力和提高死亡率起作用，外来入侵通过竞争或掠食改变营养级传导。
5. 所有状态变量均归一化处理，初值在 1 附近代表政策启动前后的相对基准量。

四、符号说明

表 1 主要符号

符号	含义
$R_j(t)$	第 j 类底层资源：水草、浮游植物、浮游动物、底栖饵料
$C_j(t)$	第 j 类四大家鱼功能群：草鱼、鲢鱼、鳙鱼、青鱼
$M(t)$	中层共享猎物代理：中小型鱼类、幼鱼与鱼虾饵料
$D(t), A(t),$ $V(t)$	分别表示长江江豚、长江鲟和外来入侵强度
$B_{\text{carp}}(t),$ $R_{\text{basal}}(t)$	分别表示四大家鱼总量与底层资源总量
$\text{CPUE}^*(t)$	四大家鱼单网次捕获量代理（权重 1,1,1,0.7）
$F, u,$ F_0	分别表示捕捞死亡率、污染强度和反事实捕捞压力
δ_B, γ_B	通道阻隔损失及长江鲟敏感性放大因子
$k_M, R_A,$ K	分别表示中层猎物承载力基准、人工放流强度和三级食物链承载力
$\Phi(t)$	食压指数，刻画消费者对底层资源的相对压力
λ_{max}	最大李雅普诺夫指数
$E(t)$	综合生态状态指数

五、问题一的模型建立与求解

5.1 模型建立

问题一先在流域平均单区内描述四类资源和四大家鱼。2025 年四大家鱼恢复约 1.8 倍约束总量尺度，湖北段单网次捕获量提升 120% 保留为局地对照。当前资料能够约束的是全流域平均恢复量，因此禁渔效应先在统一尺度上计算。观测事实与模型变量的对应关系见表 2。

为识别不同资源层的承压差异，我们保留四类资源与四类鱼的功能群对应关系，而不把四大家鱼合并为单一消费者。草鱼摄食水草、鲢鱼滤食浮游植物、鳙鱼滤食浮游动物、青鱼摄食底栖饵料，因而底层资源不能压缩为单一总量。据此建立资源–功能群配对模型，将每类鱼与对应资源一一耦合：

$$\frac{dR_j}{dt} = r_j R_j \left(1 - \frac{R_j}{K_j}\right) - \alpha_j \frac{R_j C_j}{1 + h_j R_j} - \eta_j u R_j, \quad j = 1, \dots, 4, \quad (1)$$

$$\frac{dC_j}{dt} = e_j \alpha_j \frac{R_j C_j}{1 + h_j R_j} - (m_j + F) C_j. \quad (2)$$

其中 $j = 1, 2, 3, 4$ 分别对应草鱼–水草、鲢鱼–浮游植物、鳙鱼–浮游动物、青鱼–底栖饵料。式中 $\eta_j u R_j$ 表示污染对底层资源的额外损失， F 为捕捞死亡率（禁渔情景取 $F = 0$ ，对照情景取 $F = F_0 > 0$ ）。后文统一记 $(R_1, R_2, R_3, R_4) = (W, Ph, Z, N)$ ， $(C_1, C_2, C_3, C_4) =$

表 2 问题一的观测约束与变量映射

事实来源	观测口径	对应模型量	用途
题目与公报	2025 年四大家鱼 恢复约 1.8 倍	$B_{\text{carp}}(t)$	参数标定约束
题目与公报	湖北段单网次捕 获量提升 120%	$\text{CPUE}^*(t)$	局地对照
附件 2	草鱼、鲢鱼、鳙 鱼、青鱼食性分 工	$C_1, \dots, C_4$ 与 $R_1, \dots, R_4$	机理约束
附件 3	2022 年公报中的 资源恢复事实	初值与时间基准	规模标定

(G, L, Y, Q) 。完整默认参数、场景控制量与初值模板列于附录表 13。为量化鱼类恢复对底层资源的累积压力，定义食压指数

$$\Phi(t) = \frac{C_1(t) + C_2(t) + C_3(t) + C_4(t)}{R_1(t) + R_2(t) + R_3(t) + R_4(t)}. \quad (3)$$

同时定义四大家鱼总量 $B_{\text{carp}}(t) = \sum_{j=1}^4 C_j(t)$ 、底层资源总量 $R_{\text{basal}}(t) = \sum_{j=1}^4 R_j(t)$ ，以及可观测的单网次捕获量代理

$$\text{CPUE}^*(t) = C_1(t) + C_2(t) + C_3(t) + 0.7C_4(t), \quad (4)$$

青鱼权重取 0.7，考虑其偏底栖，在单网次观测中的可见性通常低于草鱼、鲢鱼和鳙鱼。该权重仅作经验设定，不纳入参数估计。将其在 $[0.5, 1.0]$ 范围内扫描后，2025 年 CPUE^* 倍率位于 $[1.805, 1.862]$ ，主结论不变。 $\Phi(t)$ 描述资源承压过程； $\text{CPUE}^*(t)$ 用于对照湖北段“单网次捕获量提升 120%”这一局地事实^[5-6]。

5.2 参数标定与结果对照

2025 年四大家鱼约恢复 1.8 倍这一事实主要用于约束总量尺度。我们在这一问里有意把标定目标压缩到单一总量事实上。考虑到单一总量事实不足以同时识别多组结构参数，我们固定单区食物网的结构参数，只对增益缩放因子 gain_scale 做单参数标定，并保持鱼类内部比例不变； $\text{consumer_scale} = 0.60$ 仅承担鱼类初值规范化任务。

为了让 2025 年总量约束只作用于禁渔路径，我们让 F_0 不进入参数搜索，而在禁渔基线确定后单独构造“不禁渔”情景，同时把湖北段 CPUE^* 保留为标定后的局地对照。数值求解采用 `solve_ivp`，标定结果为 $\text{gain_scale} = 1.45$ ，对应 2025 年四大家鱼总量 1.805 倍、 CPUE^* 1.838 倍，对 1.8 倍与 2.2 倍的相对误差分别为 0.26% 和 16.47%。

不禁渔情景由固定反事实参数 $F_0 = 0.25$ 给出；该值位于预设区间 $[0.10, 0.40]$ 中部，只用于比较禁渔与持续捕捞的方向差异。进一步将 F_0 在该区间内等距取 7 个值重复对照，禁渔情景始终优于不禁渔情景（ B_{carp} 差值均为正），且 $F_0 \geq 0.15$ 时差值超过 30%。因此，“禁渔优于不禁渔”的判断不依赖某一个特定的 F_0 取值。

表 3 问题一模型参数的角色与来源说明

参数	数值	来源/依据	角色
结构参数（归一化单区设定）			
r_1, r_2, r_3, r_4	1.15, 0.95, 0.82, 0.72	表示四类资源的相对恢复快慢，用于保持营 养级时间尺度分离。	时间尺度约 束
K_1, K_2, K_3, K_4	(1, 1, 1, 1)	统一取归一化承载上界 1，作为资源层的尺 度基准。	归一化基准
$\alpha_1, \alpha_2, \alpha_3, \alpha_4$	1.22, 0.92, 0.84, 0.76	Holling II 型摄食耦合强度，依次对应水草-草 鱼、浮游植物-鲢鱼、浮游动物-鳙鱼、底栖饵 料-青鱼，保持摄食项与自增殖项同量级 ^[4] 。	结构耦合
h_1, h_2, h_3, h_4	0.55, 0.50, 0.48, 0.45	控制饱和摄食拐点，使响应保持有界。	饱和约束
e_1, e_2, e_3, e_4	0.43, 0.35, 0.33, 0.30	归一化转化效率，保证消费者收益不高于资 源供给量级。	能量转化
m_1, m_2, m_3, m_4	0.16, 0.14, 0.13, 0.12	归一化自然损失系数，与转化效率共同维持 平衡数量级。	背景损失
校准与情景参数（完整默认值见附录表 13）			
<i>consumer_scale</i>	0.60	固定缩放四大家鱼初值，不参与 2025 年观测 标定。	初值规范化
<i>gain_scale</i>	1.45	由 2025 年四大家鱼总量约束单参数标定得 到。	唯一标定参 数
F_0	0.25	取区间 [0.10, 0.40] 中位值，仅用于“不禁渔” 对照。	反事实对照
CPUE* 权重	(1, 1, 1, 0.7)	青鱼在单网次指标中保守下调，不纳入参数 估计。	对照指标

5.3 结果与机制解释

表 4 给出了 2025 年的总量标定结果与局地对照结果。四大家鱼总量与题目给出的恢复信息接近；CPUE* 虽低于 2.2 的湖北段对照值，但恢复方向一致。

表 4 问题一的 2025 年参数标定结果与局地对照

指标	观测值	模拟值	相对误差	说明
四大家鱼总量倍 率	1.800	1.805	0.26%	对齐
CPUE* 倍率	2.200	1.838	16.47%	一致

两项指标的方向相同，幅度却不同。四大家鱼总生物量基本复现用于标定的 1.8 倍，表明单区模型在全流域平均尺度上与标定目标一致；CPUE* 低于湖北江段的 2.2 倍，局

地渔获口径可能带来另一种增幅。两者无法用同一误差解释。

禁渔情景下，四大家鱼总量与 CPUE* 上升，基础资源末值为初值的 0.204 倍，水草最低值出现在第 10 年；食压指数同时增至 4.213。这些读数表明，鱼群恢复与底层资源改善在模型中未同步发生。0.204 和 Φ 均仅为归一化单区指标，不代表全流域实际生物量或能量金字塔的绝对比例。 Φ 持续上升与底层资源下降共同表明，消费者增长快于资源供给；捕捞压力解除后，中层鱼类的摄食压力和种间竞争随之增强，呈现”鱼类资源恢复与基础资源承压并存”。

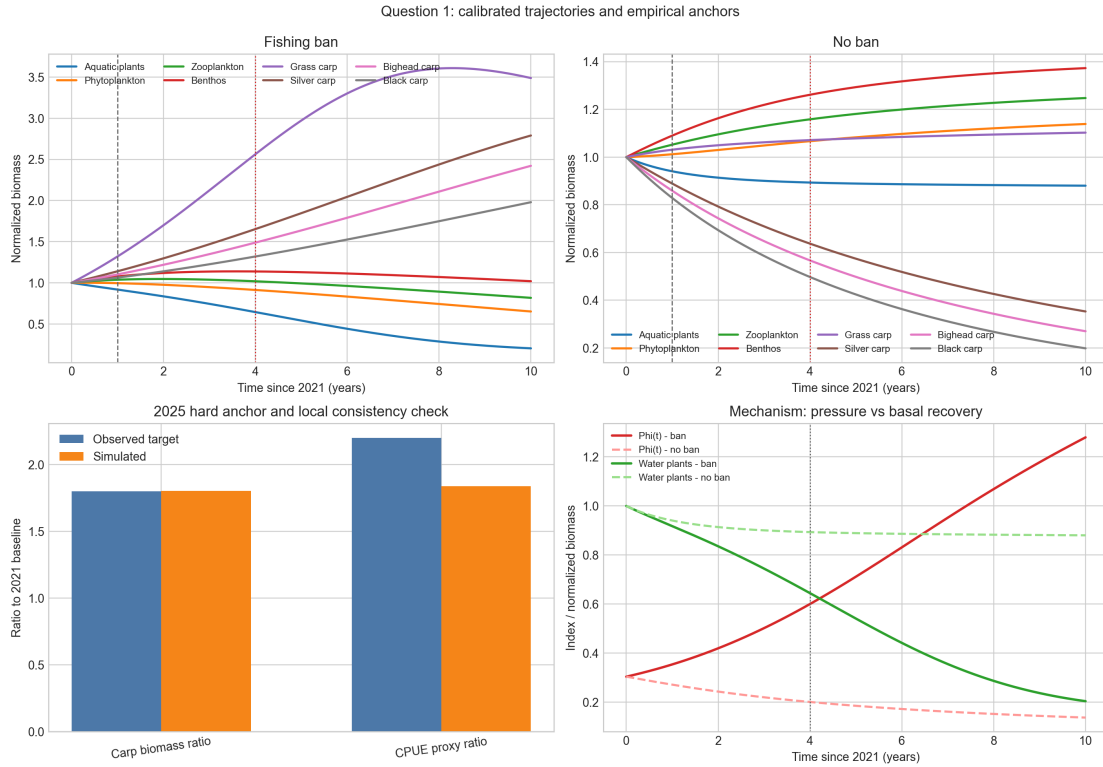


图 1 问题一：标定后的禁渔与对照情景及观测约束

六、问题二的模型建立与求解

6.1 模型建立

我们在问题二中以问题一得到的上游资源-鱼群动态轨迹作为输入，在统一初始条件下构建珍稀物种种群响应子模型，并通过对照情景分离食源供给、江湖连通性与人工放流的独立效应。其中变量 M 不再沿用四大家鱼的含义，而是代表长江江豚与长江鲟共享的中层猎物库，涵盖四大家鱼以外的中小型鱼类、幼鱼及鱼虾混合饵料； D 和 A 分别表示长江江豚与长江鲟种群， V 表示外来入侵竞争物种。

问题一的动力系统旨在刻画禁渔对基础资源层和经济鱼类的恢复过程，而问题二聚焦于珍稀保护物种可利用的公共食源库，二者研究对象与生态位并不完全对应。因此上游生态信息通过综合生态状态指数 $E(t)$ 驱动中层猎物承载力 K_M ，而非将四大家鱼的种群状态直接赋值为 M 。研究情景与模型设定的对应关系如表 5 所示。其中 $\delta_B, D(0), A(0)$

表 5 问题二的观测事实与模型映射

事实来源	观测口径	对应模型量	角色
附件 1	洄游路径、大坝阻隔、 鱼道连通性	δ_B	事实约束
附件 2	食性分工	a_D, a_A, a_V	经验参数设定
附件 2	放流补偿	R_A	情景量
附件 3 (2022 公报)	江豚 1249 头、长江鲟 438 尾	$D(0), A(0)$	初值约束
问题一输出	归一化生态指数 $E(t)$	$E(t)$	上游输入量

直接对应题目事实； a_D, a_A, a_V 为经验参数， R_A 为情景控制量（取值 0.012，对应附件 2 中长江鲟年均放流规模与 2022 公报存量的比值）， $E(t)$ 作为问题一输出的上游输入量使用。高阻隔情景提高 δ_B ，无放流情景取 $R_A = 0$ ，污染情景提高 u ，其余参数保持不变。

对任一状态量 X 记 $\tilde{X}(t) = X(t)/X(0)$ 。由问题一得到底层资源和四大家鱼轨迹后，我们构造综合生态状态指数

$$E(t) = 0.62E_{\text{basal}}(t) + 0.38E_{\text{carp}}(t), \quad (5)$$

$$E_{\text{basal}}(t) = 0.30\tilde{W}(t) + 0.25\tilde{P}h(t) + 0.25\tilde{Z}(t) + 0.20\tilde{N}(t), \quad (6)$$

$$E_{\text{carp}}(t) = 0.28\tilde{G}(t) + 0.24\tilde{L}(t) + 0.24\tilde{Y}(t) + 0.24\tilde{Q}(t). \quad (7)$$

其中 E_{basal} 保留资源端压力信号， E_{carp} 保留鱼群恢复信号。权重 0.62 和 0.38 用于同时保留这两部分信息：在问题一的标定轨迹上，2025 年 $E_{\text{basal}} = 0.904$ 、 $E_{\text{carp}} = 1.788$ ，若完全依赖鱼群层， $E(t)$ 便退化为单纯的鱼群恢复指标，从而削弱资源端信息。将 E_{basal} 权重在 $[0.50, 0.75]$ 范围内扫描后，基线情景下江豚和长江鲟末值的最大变动分别仅为 1.21% 和 0.18%，四组情景中的物种排序和作用方向保持不变。为使中层猎物承载力同时响应食源恢复和污染压力，按脚本实现取

$$K_M(E, u) = \max \{0.18, k_M(0.62 + 0.76E(t)) \max(0.30, 1 - 0.60u)\}, \quad (8)$$

其中基准取 $k_M = 1$ 。 $K_M(E, u)$ 在此作为由上游轨迹诱导的一维食源强迫项，汇集问题一中的恢复与退化信号。建立珍稀物种与入侵物种模型^[4]：

$$\frac{dM}{dt} = r_M M \left(1 - \frac{M}{K_M(E, u)}\right) - a_D \frac{MD}{1 + h_M M} - a_A \frac{MA}{1 + h_M M} - a_V \frac{MV}{1 + h_M M} - \lambda_u u M, \quad (9)$$

$$\frac{dD}{dt} = e_D a_D \frac{MD}{1 + h_M M} - (m_D + \delta_B + \mu_D u) D, \quad (10)$$

$$\frac{dA}{dt} = R_A + e_A a_A \frac{MA}{1 + h_M M} - (m_A + \gamma_B \delta_B + \mu_A u) A, \quad (11)$$

$$\frac{dV}{dt} = e_V a_V \frac{MV}{1 + h_M M} - (m_V + \mu_V u) V. \quad (12)$$

式中 δ_B 表示通道阻隔和栖息地碎片化的综合损失， R_A 表示人工放流强度， γ_B 表示长江鲟阻隔敏感性放大因子，基准取 $\gamma_B = 1.3$ 。考虑到长江鲟属洄游型鱼类，其繁殖和生长对通道连通性的依赖显著高于定居型的江豚，模型中对阻隔项作适度放大；附件 1 给出的信息显示，长江鲟洄游距离约为江豚日常活动范围的 1.3 倍，这一数值也为 γ_B 的基准取值提供了参照。将该乘数在 $[1.1, 1.5]$ 范围内扫描后，基线情景下长江鲟末值的最大变动约为 2.1%，四组情景中长江鲟的排序保持不变，说明结论对该乘数的具体取值不敏感。题目事实与模型口径的对应关系见表 5 中统一给出。

6.2 情景结果与机制讨论

在 2022 公报的归一化口径下，基线情景中江豚和长江鲟末值为 1.133 和 1.140。若只增强通道阻隔、暂不叠加额外污染，两者降至 0.901 和 0.856；将长江鲟人工放流项置为 $R_A = 0$ 时，长江鲟降至 0.519，江豚为 1.145；污染情景下两者为 0.610 和 0.704。这些数值把作用位置区分开来：通道与污染同时影响两类物种，放流变化主要落在长江鲟。表中报告终态， M 与 V 留在后文解释动态过程。无放流时长江鲟大幅下降而江豚

表 6 问题二的结果口径、相对变化与对照说明

项目	数值/情景输出	相对基线变化	是否外部验证	解释用途
2022 公报基准	江豚 1249 头、长江鲟 438 尾	—	是	作为问题二的初值与口径基准
基线情景末值	江豚 1.133，长江鲟 1.140	—	否	作为禁渔与食源修复下的基线投影
$R_A = 0$ 对照	江豚 1.145，长江鲟 0.519	江豚 +1.0%， 长江鲟 -54.5%	否	拆分自然恢复与人工放流补偿效应
高阻隔对照	江豚 0.901，长江鲟 0.856	江豚 -20.5%， 长江鲟 -24.9%	否	仅提高通道阻隔，分离生境约束效应
污染情景末值	江豚 0.610，长江鲟 0.704	江豚 -46.2%， 长江鲟 -38.2%	否	比较污染叠加阻隔对基线收益的侵蚀

变化很小，人工补偿主要作用于鲟类而不是江豚；高阻隔与污染同时压低两类物种，说明连通性和环境质量属于共同约束。模型在 $t = 3$ （对应 2024 年）给出长江鲟正增长趋势 $(A(3)/A(0) > 1)$ ，与题目所述“2024 年长江鲟自然繁殖群体重新出现产卵场”方向一致，这里仅作定性核验。

参数敏感性分析 为检验情景结论对经验参数的依赖程度，对 6 个关键参数分别施加 $\pm 20\%$ 扰动，记录终态江豚 D 和长江鲟 A 相对基线的变化率（表 7）。

表 7 问题二参数敏感性分析（ $\pm 20\%$ 扰动）

参数	$\Delta D_{+20\%}$	$\Delta D_{-20\%}$	$\Delta A_{+20\%}$	$\Delta A_{-20\%}$
δ_B （通道阻隔）	-1.9%	+2.0%	-2.5%	+2.6%
R_A （放流强度）	-0.2%	+0.2%	+10.9%	-10.9%
e_D （江豚转化率）	+8.3%	-9.4%	-0.8%	+0.9%
e_A （长江鲟转化率）	-0.0%	+0.0%	+1.8%	-1.8%
a_D （江豚摄食率）	-8.5%	+5.2%	-2.7%	+3.6%
a_A （长江鲟摄食率）	-0.8%	+0.8%	+1.7%	-1.7%

敏感性结果显示，江豚终态对 e_D 和 a_D 最敏感（ $|\Delta D| > 5\%$ ），说明江豚恢复对食物获取效率变化最为敏感。长江鲟那边最显眼的是放流强度 R_A （ $|\Delta A| \approx 11\%$ ），人工放流仍是恢复中的关键补偿项。通道阻隔 δ_B 对两物种都有负面影响，只是放到当前参数区间里看，扰动幅度还没有前两者那么大（ $< 3\%$ ）。

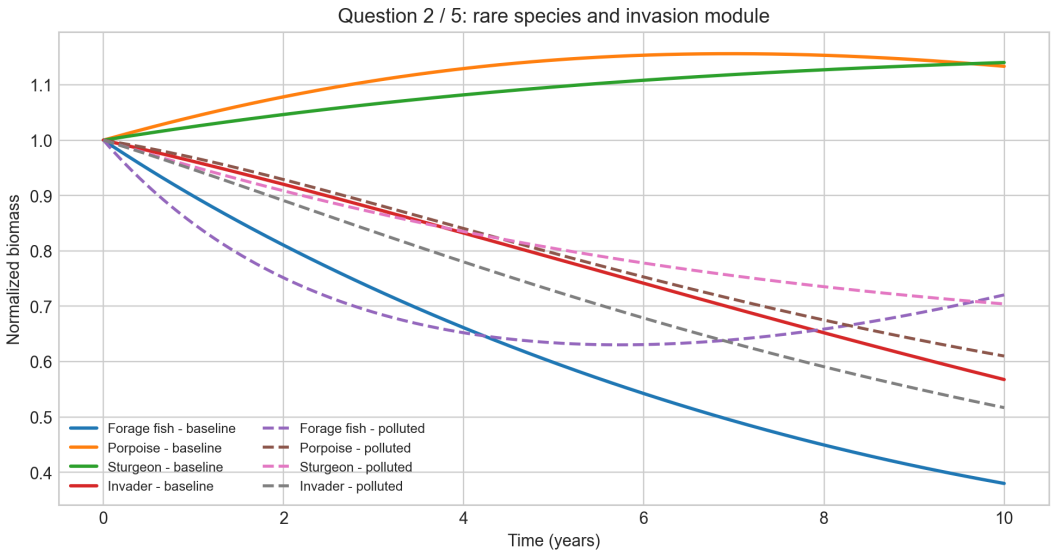


图 2 问题二和问题五共用的稀有种与入侵模块结果

七、问题三的模型建立与求解

7.1 模型建立

问题三用 Hastings-Powell 三级食物链保留底层资源、中层消费者和顶层捕食者之间的反馈，在同一参数轴上比较“资源-消费者-捕食者”反馈下的轨道类型^[7]：

$$\frac{dX_1}{dt} = rX_1 \left(1 - \frac{X_1}{K} \right) - \frac{a_1 X_1 X_2}{b_1 + X_1}, \tag{13}$$

$$\frac{dX_2}{dt} = \frac{a_1 X_1 X_2}{b_1 + X_1} - \frac{a_2 X_2 X_3}{b_2 + X_2} - d_1 X_2, \quad (14)$$

$$\frac{dX_3}{dt} = \frac{a_2 X_2 X_3}{b_2 + X_2} - d_2 X_3. \quad (15)$$

其中 K 代表基础食物链有效承载力，可汇总营养输入、水质改善和污染冲击对基础食物链的作用。物种映射上， X_1 对应底层资源层（问题一中 R_j 的聚合）， X_2 对应中层消费者（四大家鱼功能群）， X_3 对应顶层捕食者（江豚等）。这里仅沿用营养级对应关系，用来解释三级食物链的生态位；问题一的参数数值不移入本模型。

模型参数取值如表 8 所示，其中 a_1, b_1, d_1 和 a_2, b_2, d_2 分别控制中层和顶层的摄食与损失， r 固定为 1， K 作为扫描控制参数。

表 8 问题三的 Hastings-Powell 模型参数取值

参数	取值	说明
r	1.0	底层资源内禀增长率，固定为归一化基准
a_1	5.0	中层对底层的摄食率，取自 Hastings & Powell (1991) 默认值 ^[7]
b_1	1.0	中层摄食半饱和常数
a_2	0.1	顶层对中层摄食率
b_2	2.0	顶层摄食半饱和常数
d_1	0.4	中层自然死亡率
d_2	0.01	顶层自然死亡率
K	扫描	承载力控制参数，扫描范围 [1.4, 4.8]

7.2 求解与结果

参数 K 表示三级食物链的综合承载条件。固定当前参数组并取代表点后， $K \approx 1.8$ 附近对应规则有界振荡， $K \approx 2.7$ 附近对应明显周期振荡；更高 K 下轨道会呈现更不规则的波动，但严格的混沌识别留到问题四通过最大李雅普诺夫指数完成。后期统计量统一在积分后期时间窗 $t \in [90, 180]$ 上提取，包括 X_1 的峰谷差 ΔX_1 、平均峰间隔 $\bar{T}$ 和峰间隔变异系数 CV_T ，如表 9 所示。

统计量限定在积分后期时间窗内计算，以滤除初值诱发的瞬态影响。由此得到的 $\sigma(X_1)$ 、 ΔX_1 与 CV_T 才能用于不同 K 之间的轨道比较。

随着 K 增大， X_1 的标准差由 0.261 升至 0.885，峰谷差由 0.991 扩大到 4.093，而峰间隔变异系数由 0.004 升至 2.746，说明系统从规则振荡逐步走向更复杂的长期行为。第三行仅作为复杂非线性波动样本展示，不在问题三中单独给出混沌判断；相关判据仍由问题四的李雅普诺夫扫描提供。图 3 给出三类代表轨道。

表 9 问题三代表轨道的积分后期描述量

类型	K	$\sigma(X_1)$	ΔX_1	N_{peak}	$\bar{T}$	CV_T
规则有界振荡	1.795	0.261	0.991	4	20.720	0.004
明显周期振荡	2.665	0.502	2.092	3	34.178	0.014
复杂非线性波动	4.800	0.885	4.093	13	4.298	2.746

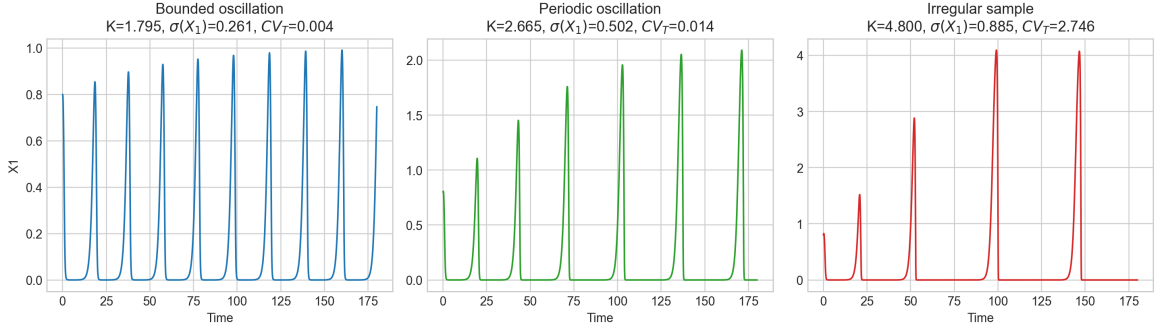


图 3 问题三代表轨道示意

八、问题四的模型建立与求解

8.1 模型建立

我们在问题四中沿用问题三的方程组和参数集，仅将判别工具替换为 Benettin 型有限时间最大李雅普诺夫指数 $\lambda_{\max}$ 。数值实现中，先由 Hastings-Powell 方程写出解析 Jacobian，再将状态方程与切向方程用四阶 Runge-Kutta 同步积分，切向向量在每个时间步进行一次归一化；“瞬态截断 40”指前 40 个时间单位的轨道不进入指数累计。基准设置取初值 $(0.8, 0.2, 0.2)$ 、步长 $dt = 0.01$ 和总积分时长 90；同时将总积分时长扩展到 150 和 200，并在 $K = 3.20$ 与 $K = 3.22$ 附近进行局部步长和初值复核，用以检验正指数起点对数值设置的敏感性。

8.2 求解与结果

基准时间窗下，先在 1.4–4.8 的粗网格和 3.0–3.4 的加密网格上扫描 $\lambda_{\max}$ ，得到首个连续正区间出现在 $K = 3.2186$ 附近，并在 3.22–3.40 区间保持为正。表 10 给出的局部复核表明，在总积分时长固定为 90 的前提下， $K = 3.20$ 与 $K = 3.22$ 之间的符号跨越对相邻步长和微扰初值仍可保留。

积分时长拉长后，正指数区间明显左移。总时长为 150 时，3.0–3.4 加密区间内各点均为正；总时长为 200 时，连续正区间从 3.11 开始，且 $\lambda_{\max}(3.20) = 0.00579$ 、 $\lambda_{\max}(3.22) = 0.00701$ 两点均为正。扩展到 5 组初值、5 个步长和 3 个积分时长的 75 组组合后，仅有 5 组（6.7%）保留 $\lambda_{\max}(3.20) < 0 < \lambda_{\max}(3.22)$ 。这不能支持稳定临界点的解释，即该符号跨越在扩展设置下不具备统计稳健性。

基准短时间窗在 $K = 3.2186$ 附近出现由负转正的局部跨越；积分时长和数值设置

表 10 问题四短时间窗局部复核

设置	$\lambda_{\max}(3.20)$	$\lambda_{\max}(3.22)$	判别	说明
基准步长 $dt = 0.010$	-0.00135	0.00117	符号跨越	3.22 附近进入正指数区间
较小步长 $dt = 0.008$	-0.00134	0.00117	符号跨越	结果与基准设置一致
较大步长 $dt = 0.012$	-0.00129	0.00123	符号跨越	结果与基准设置一致
微扰初值 (0.8005, 0.1995, 0.2)	-0.00087	0.00165	符号跨越	微扰后仍保留同样的符号关系

改变后，跨越位置随之移动，因此 $K = 3.2186$ 不能作为唯一临界点。本文只保留“承载力提高会增强系统复杂性”这一方向性结论。若需精确定位，还须采用更长积分时长 (≥ 500) 并与分岔图交叉核对，图4给出不同积分时长下的扫描结果。

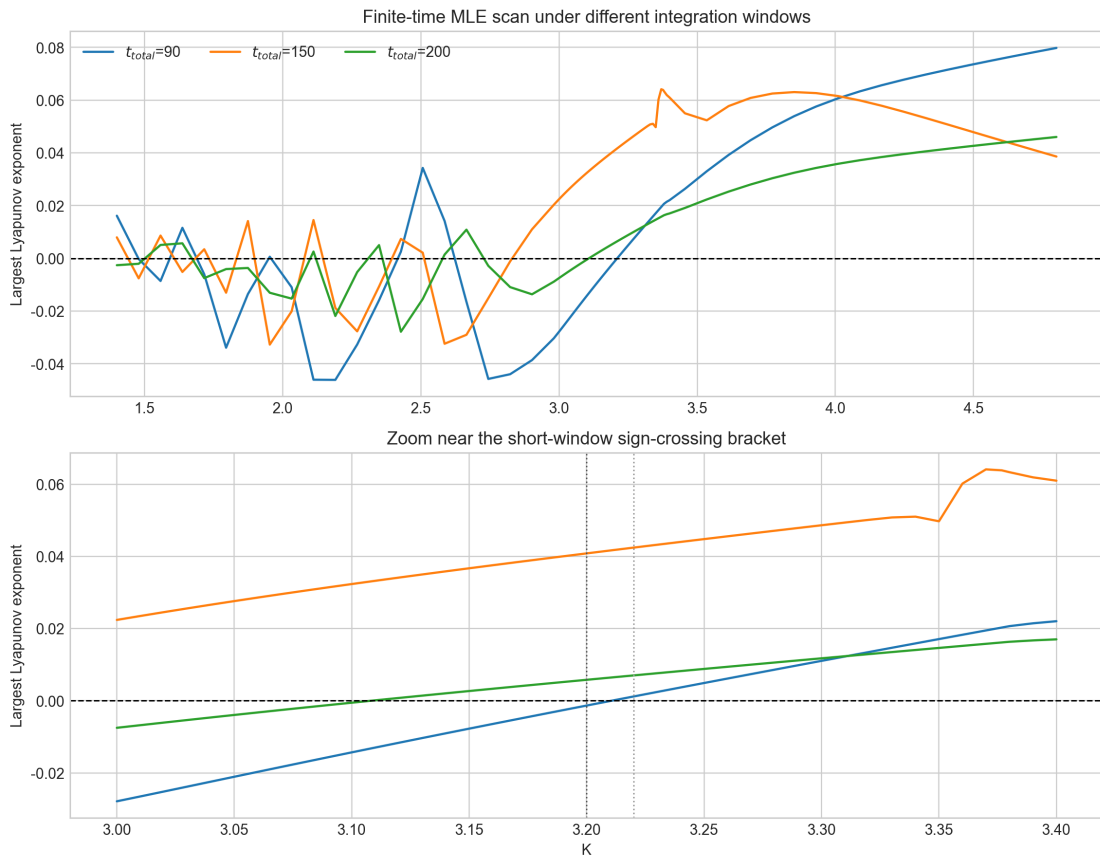


图 4 问题四不同积分时长下的有限时间李雅普诺夫指数扫描

九、问题五的模型建立与求解

9.1 模型建立

问题五沿用问题二的状态方程，污染强度、入侵初值和竞争强度需要按情景调整。问题一的基线轨迹对应 $u = 0$ ；污染情景则以 $u > 0$ 重新积分，得到与污染设定一致的 E_{basal} 和 E_{carp} ，再接入问题二模块。

考虑到污染和入侵会改变尾窗内的波动幅度，我们统一比较积分后期最后 200 个离散采样点上的平均指标，以减弱瞬时相位的影响。设第 i 个情景在该尾窗上的四个平均指标分别为 $\bar{M}_i, \bar{D}_i, \bar{A}_i, \bar{V}_i$ ，其中食源、江豚和长江鲟为效益型指标，入侵压力为成本型指标。三种情景先构造正向化指标矩阵，随后分别计算熵权和 CRITIC 权重，并取两者平均作为组合权重。记组合权重为 $w = (w_1, w_2, w_3, w_4)$ ，则综合得分定义为

$$\mathcal{H}_i = \sum_{j=1}^4 w_j z_{ij}, \quad \sum_{j=1}^4 w_j = 1, \quad w_j \geq 0. \quad (16)$$

在当前数据下，组合权重为 (0.232, 0.289, 0.298, 0.181)，分别对应食源、江豚、长江鲟和入侵压力。该指标用于比较三种情景的相对状态，不对应外部生态健康的绝对量值。

9.2 求解与结果

三种情景“仅禁渔”“禁渔+污染”“禁渔+污染+入侵”的后期平均指标列于表11。组合赋权得分依次为 0.662、0.423 和 0.155。污染情景的食源指标由 0.407 升至 0.692，江豚和长江鲟却降至 0.636 和 0.716；食源上升并不伴随珍稀物种恢复。污染上再叠加入侵后，入侵压力升至 0.641，综合得分继续下降，各指标并不同步。污染情景中食源指

表 11 问题五的积分后期平均指标与综合得分

情景	食源	江豚	长江鲟	入侵压力	$\mathcal{H}$
仅禁渔	0.407	1.143	1.135	0.602	0.662
禁渔+污染	0.692	0.636	0.716	0.546	0.423
禁渔+污染+入侵	0.598	0.624	0.711	0.641	0.155

标上升而江豚、长江鲟下降。方程中的 u 压低 K_M ，污染死亡项又造成捕食者数量下降，在当前参数组的尾窗内，捕食者减少带来的释放超过了承载损失，于是 M 平均值短时回升。这一回升仅对应模型内的过程量再分配，不表示污染改善了系统健康。

各赋权方案的排序相同。组合权重经 2000 次 [0.85, 1.15] 重归一扰动，完整排序稳定率和“入侵情景最差”稳定率均为 100%。赋权变化没有改变排序，因此这里只说明排序不随这些赋权和扰动改变。

表 12 问题五的赋权结果与排序稳健性

方案	权重 (M, D, A, V)	排序结果	说明
熵权	(0.144, 0.337, 0.351, 0.168)	仅禁渔 > 禁渔 + 污染 > 禁渔 + 污染 + 入侵	根据指标离散程度确定权重
CRITIC	(0.319, 0.242, 0.244, 0.194)	仅禁渔 > 禁渔 + 污染 > 禁渔 + 污染 + 入侵	同时考虑对比强度与相关性
组合权重	(0.232, 0.289, 0.298, 0.181)	仅禁渔 > 禁渔 + 污染 > 禁渔 + 污染 + 入侵	取熵权与 CRITIC 权重的平均值
等权对照	(0.250, 0.250, 0.250, 0.250)	仅禁渔 > 禁渔 + 污染 > 禁渔 + 污染 + 入侵	用于检查排序是否依赖某一套客观权重
局部扰动	—	仅禁渔 > 禁渔 + 污染 > 禁渔 + 污染 + 入侵	围绕组合权重做 2000 次 [0.85, 1.15] 重归一扰动，完整排序稳定率为 100%

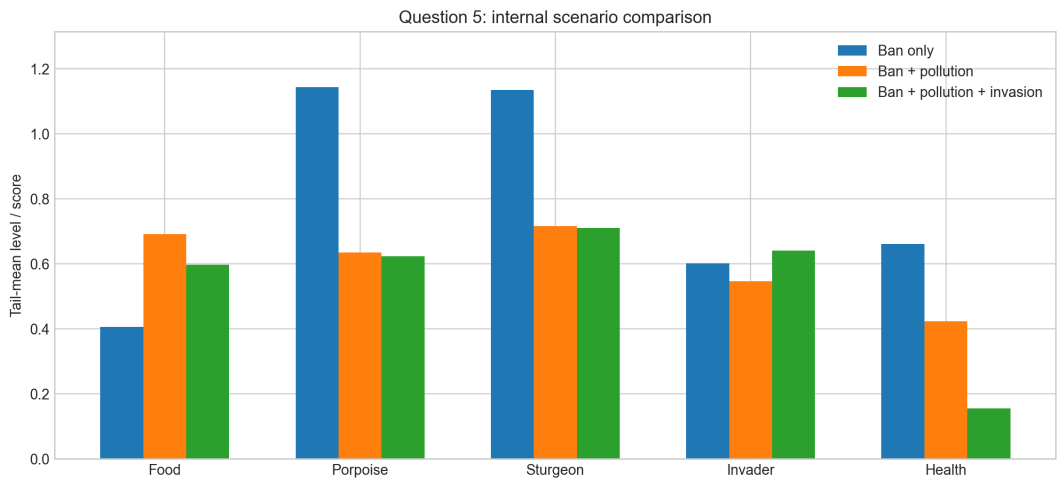


图 5 问题五的情景比较结果

十、治理建议

问题一中四大家鱼总量回升时，水草、浮游动物和底栖饵料仍可能下降；问题二中江豚与长江鲟对通道和放流的响应又不相同。治理重点不能只落在捕捞总量，还包括资源层和物种受限环节。监测同时保留四大家鱼总量、底层资源和食压指数，不只报告鱼类总量；鱼道、产卵场连通和放流节律也不按同一方案处理。

问题五的污染情景中，部分过程量短时上升，珍稀物种终态和综合得分却下降。方向相反，所以局部过程改善并不等于系统整体好转。对鄱阳湖等支流湖泊，可将其作为高风险前沿，优先布设入侵物种早期监测、拦截和清除，并把水质、连通性和放流措施纳入同一比较。这里的模型给出措施先后；如果把情景排序直接解释为实际治理效果，结论不在模型能够支持的范围内。

十一、结论

模型中的四大家鱼恢复到禁渔前约 1.8 倍，底层资源末值却降至 0.204，食压指数持续上升。恢复主要积聚在消费者层，资源供给并未同步增加；这些数值仅表示归一化单区系统中的相对压力，不对应全流域实测生物量结构。恢复收益仍能传到长江江豚和长江鲟，只是这条路径并不稳定：通道阻隔会同时削弱两者，人工放流的补偿更多落在长江鲟。

承载力提高后，三级食物链不会一直维持规则振荡，长期波动随之复杂。有限时间最大李雅普诺夫指数支持这一变化方向，但还不足以确定唯一临界点； K 也只表示归一化结构压力，而非长江某一河段的实测承载量。污染和入侵情景中，食源过程量可能与珍稀物种终态分离。禁渔只是恢复链条的起点；现有计算不能给出治理措施的实际效果，也不能由此断定恢复收益会长期留在系统内部。是否留得住，还取决于水质、连通性、放流和入侵控制。

十二、模型评价与改进方向

12.1 单区食物网的解释边界

单区食物网在流域平均尺度上能够同时容纳总量标定、局地对照和反事实比较。我们的标定结果下，2025 年四大家鱼总量为 1.805，CPUE 代理为 1.838；对 *gain_scale*、*consumer_scale* 和 F_0 做 $\pm 10\%$ 扰动后，恢复方向和禁渔占优关系保持不变。进一步对资源增长、摄食耦合、转化效率和自然损失四组结构参数做 $\pm 20\%$ 分组扰动，并在每组扰动后重新标定总量目标，CPUE 代理仍位于 1.824–1.840 区间。这一结构按食性组成相互并联的资源–鱼类关系：四类资源分别与四类鱼直接耦合，用于识别哪一层首先承压，而资源间竞争与鱼类交叉摄食没有显式展开。

CPUE* 权重仍带有经验成分，湖北段数据也具有局地网具口径。按现有设定，模型能够稳健支持的是“全流域平均恢复”和“局地恢复方向一致”，还不足以精确解释湖北段增幅。若要提高解释力，标准化 CPUE 和连续河段监测是前提；取得这些数据后，再引入横向竞争和交叉摄食，而不是只增加状态维度。

12.2 珍稀物种响应的归因能力

问题二把公报初值、通道阻隔、放流补偿和上游食源纳入同一响应框架后，连通性约束与工程补偿的异质作用得以区分。高阻隔会使江豚和长江鲟末值降至 0.901 和 0.856，无放流时长江鲟降至 0.519；配合 $\pm 20\%$ 参数敏感性结果看，江豚对自身摄食与转化参数更敏感，长江鲟则对放流强度更敏感。

这一部分的限制主要来自状态结构的简化。当前模型尚未展开分龄、繁殖季节和分阶段放流调度，也把江豚、长江鲟与入侵者对中层猎物的半饱和响应压缩在同一尺度上。按这一级别的简化，它更适合比较不同机制的相对强弱，而不直接外推为精细的长期种群预测。

12.3 长期行为展示的适用范围

Hastings-Powell 降维模型保留了“资源-消费者-捕食者”的最小反馈闭环，使承载力变化与轨道类型之间的关系可以在单一参数轴上比较。更关键的是，在这一层抽象里轨道形态确实会随 K 改变。代表点 $K = 1.795, 2.665, 4.800$ 的后期统计量表明，随着 K 增大，波动幅度和峰间隔离散程度同步放大，复杂非线性波动并非单纯来自图像印象。

这一模型承担的是长期行为展示，而不是河段尺度的参数映射。 X_1, X_2, X_3 与前文功能群只保持营养级对应关系，不继承问题一的具体参数口径，因此它适合解释行为类型如何变化，而不适合反推具体河段的承载阈值；文中的 K 只对应归一化结构压力水平。

12.4 复杂化判据的稳健范围

有限时间最大李雅普诺夫指数把轨道复杂化转化为可比较的数值判据。基准设置在 $K = 3.2186$ 附近出现局部由负转正的跨越，短时间窗内对相邻步长和局部初值仍可复现。

一旦将积分时长扩展至 150 和 200，正指数区间明显左移；75 组扩展测试中仅有 5 组保留 $\lambda_{\max}(3.20) < 0 < \lambda_{\max}(3.22)$ 。因此，这些测试支持“复杂性随承载力提高而增强”的判断，而不把 3.2186 写成唯一阈值。精细分岔定位仍有待更长积分时长、致密网格与分岔图交叉验证。

12.5 综合评价的排序可信度

问题五的综合评价只用于三种情景之间的相对比较。四个指标经正向化后，熵权、CRITIC、等权和 2000 次局部扰动都给出同一排序，其中“禁渔 + 污染 + 入侵”始终最差，当前比较范围内，复合压力情景始终排在末位。

综合分数不能替代单指标解释。食源、江豚和长江鲟仍对应不同层面的生态状态；若后续补入独立生态监测量并扩展污染、入侵强度梯度，新增数据用于检查这一排序能否外推。

沿“恢复收益-上传导-长期复杂化-复合压力检验”关系，问题一、二分别给出恢复收益出现的位置和向珍稀物种传递时的损失；问题三、四把承载条件与长期轨道联系起来；问题五再比较污染、入侵叠加后的相对状态。各模块共用时间口径、情景设定和上游输入，承担不同判断，彼此不替代。

现有资料可以支持方向性的情景比较和模型内机制解释。若连续河段监测、标准化 CPUE、珍稀物种年度调查及污染分级资料齐全，单区结果才能按河段复算，长期动力学参数也才能与实际营养级数据对应。

十三、参考文献

- [1] 农业农村部长江流域渔政监督管理办公室, 水利部长江水利委员会, 生态环境部长江流域生态环境监督管理局, 等. 长江流域水生生物资源及生境状况公报 (2022 年) [R/OL]. 农业农村部长江流域渔政监督管理办公室; 水利部长江水利委员会; 生态环境

- 境部长江流域生态环境监督管理局; 交通运输部长江航务管理局. 2023 [2026-05-12]. https://www.gov.cn/lianbo/bumen/202310/content_6908750.htm.
- [2] 农业农村部长江流域渔政监督管理办公室, 水利部长江水利委员会, 生态环境部长江流域生态环境监督管理局, 等. 长江流域水生生物资源及生境状况公报 (2023 年) [R/OL]. 农业农村部长江流域渔政监督管理办公室; 水利部长江水利委员会; 生态环境部长江流域生态环境监督管理局; 交通运输部长江航务管理局. 2024 [2026-05-12]. https://www.gov.cn/lianbo/bumen/202408/content_6967878.htm.
- [3] 2026 广州高校数学建模邀请赛组委会. 2026 广州高校数学建模邀请赛 A 题: 长江流域人类活动对长江渔业生态的影响[Z]. 2026.
- [4] HOLLING C S. Some characteristics of simple types of predation and parasitism[J/OL]. The Canadian Entomologist, 1959, 91(7): 385-398. DOI: 10.4039/ent91385-7.
- [5] MAUNDER M N, PUNT A E. Standardizing catch and effort data: a review of recent approaches[J/OL]. Fisheries Research, 2004, 70(2-3): 141-159. DOI: 10.1016/j.fishres.2004.08.002.
- [6] MAUNDER M N, SIBERT J R, FONTENEAU A, et al. Interpreting catch per unit effort data to assess the status of individual stocks and communities[J/OL]. ICES Journal of Marine Science, 2006, 63(8): 1373-1385. DOI: 10.1016/j.icesjms.2006.05.008.
- [7] HASTINGS A, POWELL T. Chaos in a three-species food chain[J/OL]. Ecology, 1991, 72(3): 896-903. DOI: 10.2307/1940591.

附 录

附录 1: 问题一完整默认参数附表

表 13 问题一完整默认参数附表

参数	默认值	依据/说明	角色
资源增长与归一化基准			
r_1, r_2, r_3, r_4	1.15, 0.95, 0.82, 0.72	归一化相对恢复速度排序，表示水草快于浮游植物、浮游植物快于浮游动物、浮游动物快于底栖饵料；用于维持营养级时间尺度分离。	时间尺度约束
K_1, K_2, K_3, K_4	(1, 1, 1, 1)	统一固定为归一化承载上界，不对应河段实测承载量。	归一化基准
摄食耦合与饱和响应			
$\alpha_1, \alpha_2, \alpha_3, \alpha_4$	1.22, 0.92, 0.84, 0.76	Holling II 型摄食耦合强度，按 $j = 1, 2, 3, 4$ 分别对应水草-草鱼、浮游植物-鲢鱼、浮游动物-鳙鱼、底栖饵料-青鱼 ^[4] 。	结构耦合
h_1, h_2, h_3, h_4	0.55, 0.50, 0.48, 0.45	饱和摄食拐点参数，使系统在归一化尺度下保持有界响应。	饱和约束
转化效率与自然损失			
e_1, e_2, e_3, e_4	0.43, 0.35, 0.33, 0.30	归一化转化效率；沿“草鱼-水草、鲢鱼-浮游植物、鳙鱼-浮游动物、青鱼-底栖饵料”顺序设置为递减，避免消费者收益超过资源层可供给量级。	能量转化
m_1, m_2, m_3, m_4	0.16, 0.14, 0.13, 0.12	归一化自然损失系数，与转化效率共同维持消费者层在禁渔和非禁渔情景下的数量级稳定。	背景损失
校准、反事实与场景控制			
$consumer_scale$	0.60	固定缩放四大家鱼初值，保留既有物种占比，不由 2025 年观测标定估计。	初值规范化
$gain_scale$	1.45	由 2025 年“四大家鱼恢复到禁渔前约 1.8 倍”这一总量约束单参数搜索得到。	唯一标定参数
F （禁渔情景）	0	禁渔基线情景下共同捕捞压力取零；这里给出的是场景取值，而非新增独立参数。	基线政策设定
F_0	0.25	不禁渔反事实情景的固定共同捕捞压力，取自区间 [0.10, 0.40] 的中位值，不参与观测校准。	反事实对照
u	0	问题一中污染强度默认取零，避免与问题五的污染情景混用。	基线环境设定
观测代理与初值模板			
CPUE* 权重	(1, 1, 1, 0.7) (0.88, 0.82, 0.76,	草鱼、鲢鱼、鳙鱼等权；青鱼因偏底栖、单网次可见性较低而保守下调。	捕获量代理
y_0^{base}	0.70, 0.46, 0.42) 0.38, 0.34)	归一化初值模板，资源四维在前、鱼类四维在后；四类鱼在仿真前统一乘以 $consumer_scale = 0.60$ 。	状态初始化

附录 2：完整 Python 源程序

```
from __future__ import annotations

import json
import math
from dataclasses import dataclass
from datetime import datetime
from pathlib import Path
from typing import Callable

import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt
import numpy as np
from scipy.integrate import solve_ivp
from scipy.interpolate import interp1d

ROOT = Path(__file__).resolve().parents[1]
FIG_DIR = ROOT / "figures"
```

```

OUT_DIR = ROOT / "outputs"
FIG_DIR.mkdir(parents=True, exist_ok=True)
OUT_DIR.mkdir(parents=True, exist_ok=True)

plt.style.use("seaborn-v0_8-whitegrid")
plt.rcParams.update(
    {
        "figure.dpi": 160,
        "savefig.dpi": 220,
        "axes.unicode_minus": False,
        "font.size": 10,
    }
)

def _clip_positive(x: np.ndarray) -> np.ndarray:
    return np.maximum(x, 0.0)

def _normalize(series: np.ndarray, ref: float | None = None) -> np.ndarray:
    base = series[0] if ref is None else ref
    return series / (base if base != 0 else 1.0)

def _safe_savefig(fig: plt.Figure, path: Path) -> None:
    try:
        fig.savefig(path, bbox_inches="tight")
    except PermissionError:
        fallback = Path.cwd() / f"{path.stem}_refresh_{datetime.now().strftime('%Y%m%d_%H%M%S')}-{path.suffix}"
        fig.savefig(fallback, bbox_inches="tight")
        print(f"[warn] locked figure path: {path}; wrote {fallback.name} instead")

def _positive_normalize(weights: np.ndarray) -> np.ndarray:
    weights = np.clip(np.array(weights, dtype=float), 0.0, None)
    total = float(np.sum(weights))
    if total <= 1e-12:
        return np.full_like(weights, 1.0 / len(weights))
    return weights / total

def _normalize_indicator_matrix(matrix: np.ndarray, benefit_mask: np.ndarray) -> np.ndarray:
    matrix = np.array(matrix, dtype=float)
    out = np.zeros_like(matrix, dtype=float)
    for j in range(matrix.shape[1]):
        col = matrix[:, j]
        span = float(np.ptp(col))
        if span <= 1e-12:
            out[:, j] = 1.0
            continue
        if benefit_mask[j]:
            out[:, j] = (col - np.min(col)) / span
        else:
            out[:, j] = (np.max(col) - col) / span
    return out

def entropy_weights(matrix: np.ndarray) -> np.ndarray:
    matrix = np.array(matrix, dtype=float)
    m, n = matrix.shape
    if m <= 1:

```

```

        return np.full(n, 1.0 / n)
    col_sums = np.sum(matrix, axis=0, keepdims=True)
    probs = np.divide(matrix, col_sums, out=np.full_like(matrix, 1.0 / m), where=col_sums > 1e-12)
    with np.errstate(divide="ignore", invalid="ignore"):
        terms = np.where(probs > 0, probs * np.log(probs), 0.0)
    entropy = -np.sum(terms, axis=0) / math.log(m)
    diversification = 1.0 - entropy
    return _positive_normalize(diversification)

def critic_weights(matrix: np.ndarray) -> np.ndarray:
    matrix = np.array(matrix, dtype=float)
    std = np.std(matrix, axis=0, ddof=0)
    if np.all(std <= 1e-12):
        return np.full(matrix.shape[1], 1.0 / matrix.shape[1])
    corr = np.corrcoef(matrix, rowvar=False)
    corr = np.nan_to_num(corr, nan=0.0)
    np.fill_diagonal(corr, 1.0)
    contrast = std * np.sum(1.0 - corr, axis=1)
    return _positive_normalize(contrast)

Q2_PROXY_BASAL_WEIGHT = 0.62
Q2_PROXY_WEIGHT_SCAN = np.linspace(0.50, 0.75, 6)
Q2_STURGEON_BARRIER_MULTIPLIER = 1.30
Q2_STURGEON_BARRIER_SCAN = np.linspace(1.10, 1.50, 5)
Q2_DIRECTIONAL_VALIDATION_T = 3.0

def fishing_pressure(t: float, h0: float) -> float:
    return h0

@dataclass(frozen=True)
class FoodWebParams:
    """Dimensionless structural parameters for the normalized single-zone food-web model.

    Only ``gain_scale`` is calibrated against the 2025 aggregate-carp anchor.
    The remaining coefficients are structural scenario settings used to preserve
    relative trophic time scales and Holling-type couplings in the normalized system.
    """
    rw: float = 1.15
    rp: float = 0.95
    rz: float = 0.82
    rn: float = 0.72
    kw: float = 1.0
    kp: float = 1.0
    kz: float = 1.0
    kn: float = 1.0
    a_wg: float = 1.22
    a_ps: float = 0.92
    a_zb: float = 0.84
    a_nq: float = 0.76
    h_w: float = 0.55
    h_p: float = 0.50
    h_z: float = 0.48
    h_n: float = 0.45
    e_wg: float = 0.43
    e_ps: float = 0.35
    e_zb: float = 0.33
    e_nq: float = 0.30
    m_g: float = 0.16

```

```

m_s: float = 0.14
m_b: float = 0.13
m_q: float = 0.12
gain_scale: float = 1.0
h_fishing: float = 0.0
pollution: float = 0.0

def food_web_rhs(t: float, y: np.ndarray, p: FoodWebParams) -> np.ndarray:
    w, ph, z, n, g, s, b, q = y
    h = fishing_pressure(t, p.h_fishing)
    u = p.pollution

    env = max(0.18, 1.0 - 0.55 * u)
    dw = p.rw * w * (1.0 - w / p.kw) * env - p.a_wg * w * g / (1.0 + p.h_w * w) - 0.14 * u * w
    dph = p.rp * ph * (1.0 - ph / p.kp) * env - p.a_ps * ph * s / (1.0 + p.h_p * ph) - 0.10 * u * ph
    dz = p.rz * z * (1.0 - z / p.kz) * env - p.a_zb * z * b / (1.0 + p.h_z * z) - 0.10 * u * z
    dn = p.rn * n * (1.0 - n / p.kn) * env - p.a_nq * n * q / (1.0 + p.h_n * n) - 0.08 * u * n

    dg = p.gain_scale * p.e_wg * p.a_wg * w * g / (1.0 + p.h_w * w) - (p.m_g + h + 0.12 * u) * g
    ds = p.gain_scale * p.e_ps * p.a_ps * ph * s / (1.0 + p.h_p * ph) - (p.m_s + h + 0.12 * u) * s
    db = p.gain_scale * p.e_zb * p.a_zb * z * b / (1.0 + p.h_z * z) - (p.m_b + h + 0.12 * u) * b
    dq = p.gain_scale * p.e_nq * p.a_nq * n * q / (1.0 + p.h_n * n) - (p.m_q + h + 0.12 * u) * q

    return np.array([dw, dph, dz, dn, dg, ds, db, dq], dtype=float)

@dataclass(frozen=True)
class RareSpeciesParams:
    r_m: float = 0.55
    k_m: float = 1.0
    a_d: float = 0.52
    a_s: float = 0.18
    a_v: float = 0.36
    h_m: float = 0.55
    e_d: float = 0.42
    e_s: float = 0.24
    e_v: float = 0.38
    m_d: float = 0.050
    m_s: float = 0.060
    m_v: float = 0.100
    barrier: float = 0.03
    sturgeon_barrier_multiplier: float = Q2_STURGEON_BARRIER_MULTIPLIER
    pollution: float = 0.0
    release_s: float = 0.012

def rare_species_rhs(
    t: float,
    y: np.ndarray,
    p: RareSpeciesParams,
    food_supply: Callable[[float], float],
) -> np.ndarray:
    m, d, s, v = y
    u = p.pollution
    supply = float(food_supply(t))
    k_eff = p.k_m * (0.62 + 0.76 * supply) * max(0.30, 1.0 - 0.60 * u)
    k_eff = max(k_eff, 0.18)

    dm = p.r_m * m * (1.0 - m / k_eff) - p.a_d * m * d / (1.0 + p.h_m * m) - p.a_s * m * s / (1.0 + p.h_m * m) - p
        .a_v * m * v / (1.0 + p.h_m * m) - 0.10 * u * m
    dd = p.e_d * p.a_d * m * d / (1.0 + p.h_m * m) - (p.m_d + p.barrier + 0.08 * u) * d

```

```

ds = (
    p.release_s
    + p.e_s * p.a_s * m * s / (1.0 + p.h_m * m)
    - (p.m_s + p.sturgeon_barrier_multiplier * p.barrier + 0.11 * u) * s
)
dv = p.e_v * p.a_v * m * v / (1.0 + p.h_m * m) - (p.m_v + 0.05 * u) * v
return np.array([dm, dd, ds, dv], dtype=float)

@dataclass(frozen=True)
class HPPParams:
    r: float = 1.0
    a1: float = 5.0
    b1: float = 1.0
    a2: float = 0.1
    b2: float = 2.0
    d1: float = 0.4
    d2: float = 0.01
    k: float = 3.0

def hp_rhs(t: float, y: np.ndarray, p: HPPParams) -> np.ndarray:
    x1, x2, x3 = y
    dx1 = p.r * x1 * (1.0 - x1 / p.k) - p.a1 * x1 * x2 / (p.b1 + x1)
    dx2 = p.a1 * x1 * x2 / (p.b1 + x1) - p.a2 * x2 * x3 / (p.b2 + x2) - p.d1 * x2
    dx3 = p.a2 * x2 * x3 / (p.b2 + x2) - p.d2 * x3
    return np.array([dx1, dx2, dx3], dtype=float)

def hp_jacobian(y: np.ndarray, p: HPPParams) -> np.ndarray:
    x1, x2, x3 = y
    j11 = p.r * (1.0 - 2.0 * x1 / p.k) - p.a1 * x2 * p.b1 / (p.b1 + x1) ** 2
    j12 = -p.a1 * x1 / (p.b1 + x1)
    j13 = 0.0

    j21 = p.a1 * x2 * p.b1 / (p.b1 + x1) ** 2
    j22 = p.a1 * x1 / (p.b1 + x1) - p.a2 * x3 * p.b2 / (p.b2 + x2) ** 2 - p.d1
    j23 = -p.a2 * x2 / (p.b2 + x2)

    j31 = 0.0
    j32 = p.a2 * x3 * p.b2 / (p.b2 + x2) ** 2
    j33 = p.a2 * x2 / (p.b2 + x2) - p.d2

    return np.array([[j11, j12, j13], [j21, j22, j23], [j31, j32, j33]], dtype=float)

def rk4_step_state_tangent(
    y: np.ndarray,
    v: np.ndarray,
    dt: float,
    p: HPPParams,
) -> tuple[np.ndarray, np.ndarray]:
    def f(x: np.ndarray) -> np.ndarray:
        return hp_rhs(0.0, x, p)

    def g(x: np.ndarray, vec: np.ndarray) -> np.ndarray:
        return hp_jacobian(x, p) @ vec

    k1y = f(y)
    k1v = g(y, v)

    y2 = y + 0.5 * dt * k1y

```



```

v2 = v + 0.5 * dt * k1v
k2y = f(y2)
k2v = g(y2, v2)

y3 = y + 0.5 * dt * k2y
v3 = v + 0.5 * dt * k2v
k3y = f(y3)
k3v = g(y3, v3)

y4 = y + dt * k3y
v4 = v + dt * k3v
k4y = f(y4)
k4v = g(y4, v4)

y_next = y + dt * (k1y + 2 * k2y + 2 * k3y + k4y) / 6.0
v_next = v + dt * (k1v + 2 * k2v + 2 * k3v + k4v) / 6.0
y_next = np.clip(y_next, 1e-10, None)
return y_next, v_next

def lyapunov_max(p: HPParams, y0: np.ndarray, dt: float = 0.01, t_trans: float = 50.0, t_total: float = 150.0) ->
    float:
    y = np.array(y0, dtype=float)
    v = np.array([1.0, 0.0, 0.0], dtype=float)
    steps_trans = int(t_trans / dt)
    steps = int(t_total / dt)

    for _ in range(steps_trans):
        y, v = rk4_step_state_tangent(y, v, dt, p)
        norm = np.linalg.norm(v)
        if norm <= 0:
            v = np.array([1.0, 0.0, 0.0], dtype=float)
        else:
            v /= norm

    acc = 0.0
    for _ in range(steps):
        y, v = rk4_step_state_tangent(y, v, dt, p)
        norm = np.linalg.norm(v)
        norm = max(norm, 1e-12)
        acc += math.log(norm)
        v /= norm
    return acc / (steps * dt)

def simulate_food_web(p: FoodWebParams, t_span: tuple[float, float], y0: np.ndarray, n: int = 1201):
    t_eval = np.linspace(t_span[0], t_span[1], n)
    sol = solve_ivp(lambda t, y: food_web_rhs(t, y, p), t_span, y0, t_eval=t_eval, method="RK45", rtol=1e-7, atol
        =1e-9)
    if not sol.success:
        raise RuntimeError(sol.message)
    return sol.t, np.clip(sol.y, 0.0, None)

def simulate_rare_species(
    p: RareSpeciesParams,
    t_span: tuple[float, float],
    y0: np.ndarray,
    food_supply: Callable[[float], float],
    n: int = 1201,
):
    t_eval = np.linspace(t_span[0], t_span[1], n)

```

```

sol = solve_ivp(
    lambda t, y: rare_species_rhs(t, y, p, food_supply),
    t_span,
    y0,
    t_eval=t_eval,
    method="RK45",
    rtol=1e-7,
    atol=1e-9,
)
if not sol.success:
    raise RuntimeError(sol.message)
return sol.t, np.clip(sol.y, 0.0, None)

def build_ecosystem_index_components(series: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    w, ph, z, n, g, s, b, q = series
    basal = 0.30 * _normalize(w) + 0.25 * _normalize(ph) + 0.25 * _normalize(z) + 0.20 * _normalize(n)
    carp = 0.28 * _normalize(g) + 0.24 * _normalize(s) + 0.24 * _normalize(b) + 0.24 * _normalize(q)
    return basal, carp

def build_ecosystem_index(series: np.ndarray, basal_weight: float = Q2_PROXY_BASAL_WEIGHT) -> np.ndarray:
    basal, carp = build_ecosystem_index_components(series)
    return float(basal_weight) * basal + (1.0 - float(basal_weight)) * carp

Q1_TIME_ANCHORS = {"policy_start": 2021, "gazette_year": 2022, "monitor_year": 2025}
Q1_TARGETS = {"aggregate_carp_ratio_2025": 1.8, "cpue_proxy_ratio_2025": 2.2}
# Heuristic proxy weights: black carp is down-weighted because of lower single-net visibility.
Q1_CPUE_WEIGHTS = np.array([1.0, 1.0, 1.0, 0.7], dtype=float)
# Fixed initial-state normalization for the single-zone competition paper version.
Q1_FIXED_CONSUMER_SCALE = 0.60
# Mid-range counterfactual fishing pressure used only for the no-ban scenario.
Q1_COUNTERFACTUAL_H0 = 0.25
Q1_COUNTERFACTUAL_SCAN = np.linspace(0.10, 0.40, 7)

def total_carp(series: np.ndarray) -> np.ndarray:
    return np.sum(series[4:], axis=0)

def total_basal(series: np.ndarray) -> np.ndarray:
    return np.sum(series[:4], axis=0)

def cpue_proxy(series: np.ndarray, weights: np.ndarray = Q1_CPUE_WEIGHTS) -> np.ndarray:
    return np.sum(series[4:] * weights[:, None], axis=0)

def pressure_index(series: np.ndarray) -> np.ndarray:
    resources = total_basal(series)
    consumers = total_carp(series)
    return consumers / np.maximum(resources, 1e-12)

def q2_end_values(series: np.ndarray) -> dict[str, float]:
    sturgeon_end = float(_normalize(series[2])[-1])
    return {
        "M": float(_normalize(series[0])[-1]),
        "D": float(_normalize(series[1])[-1]),
        "A": sturgeon_end,
        "S": sturgeon_end,
    }

```

```

        "V": float(_normalize(series[3])[-1]),
    }

def health_score(row: np.ndarray, weights: tuple[float, float, float, float]) -> float:
    return float(weights[0] * row[0] + weights[1] * row[1] + weights[2] * row[2] + weights[3] * row[3])

def make_q1_initial_state(base: np.ndarray, consumer_scale: float) -> np.ndarray:
    y0 = np.array(base, dtype=float)
    y0[4:] *= consumer_scale
    return y0

def q1_metrics(t: np.ndarray, series: np.ndarray) -> dict:
    time_to_value = {
        "2021": float(t[0]),
        "2022": float(Q1_TIME_ANCHORS["gazette_year"] - Q1_TIME_ANCHORS["policy_start"]),
        "2025": float(Q1_TIME_ANCHORS["monitor_year"] - Q1_TIME_ANCHORS["policy_start"]),
    }
    idx_2025 = int(np.argmin(np.abs(t - time_to_value["2025"])))
    idx_2022 = int(np.argmin(np.abs(t - time_to_value["2022"])))

    carp_total = total_carp(series)
    basal_total = total_basal(series)
    cpue = cpue_proxy(series)
    phi = pressure_index(series)
    water = _normalize(series[0])
    carp_norm = _normalize(carp_total)
    recover_mask = carp_norm > 1.01
    milestones = {
        "carp_recovers_above_baseline_year": float(t[np.argmax(recover_mask)]) if np.any(recover_mask) else None,
        "pressure_peak_year": float(t[int(np.argmax(phi))]),
        "water_min_year": float(t[int(np.argmin(water))]),
    }
    return {
        "aggregate_carp_ratio_2025": float(_normalize(carp_total)[idx_2025]),
        "aggregate_carp_ratio_2022": float(_normalize(carp_total)[idx_2022]),
        "cpue_proxy_ratio_2025": float(_normalize(cpue)[idx_2025]),
        "cpue_proxy_ratio_2022": float(_normalize(cpue)[idx_2022]),
        "pressure_start": float(phi[0]),
        "pressure_end": float(phi[-1]),
        "pressure_change": float(phi[-1] / max(phi[0], 1e-12)),
        "water_end": float(water[-1]),
        "water_min": float(np.min(water)),
        "water_min_year": float(t[int(np.argmin(water))]),
        "milestones": milestones,
        "species_end": {
            "W": float(_normalize(series[0])[-1]),
            "G": float(_normalize(series[4])[-1]),
            "S": float(_normalize(series[5])[-1]),
            "B": float(_normalize(series[6])[-1]),
            "Q": float(_normalize(series[7])[-1]),
        },
    }

def q1_loss(metrics: dict) -> float:
    agg = metrics["aggregate_carp_ratio_2025"]
    return ((agg - Q1_TARGETS["aggregate_carp_ratio_2025"]) / Q1_TARGETS["aggregate_carp_ratio_2025"]) ** 2

```

```

def calibrate_q1(
    base_y0: np.ndarray,
    t_span: tuple[float, float],
    structural_overrides: dict[str, float] | None = None,
) -> tuple[np.ndarray, FoodWebParams, dict]:
    best = None
    consumer_scale = Q1_FIXED_CONSUMER_SCALE
    gain_grid = np.linspace(0.8, 1.8, 101)
    y0 = make_q1_initial_state(base_y0, consumer_scale)
    structural_kwargs = FoodWebParams().__dict__.copy()
    if structural_overrides:
        structural_kwargs.update(structural_overrides)
    for gain_scale in gain_grid:
        params = FoodWebParams(
            **{
                **structural_kwargs,
                "h_fishing": 0.0,
                "pollution": 0.0,
                "gain_scale": float(gain_scale),
            }
        )
        t, series = simulate_food_web(params, t_span, y0, n=481)
        metrics = q1_metrics(t, series)
        loss = q1_loss(metrics)
        if best is None or loss < best["loss"]:
            best = {
                "loss": float(loss),
                "consumer_scale": float(consumer_scale),
                "gain_scale": float(gain_scale),
                "t": t,
                "series": series,
                "metrics": metrics,
            }
    assert best is not None
    best_y0 = make_q1_initial_state(base_y0, best["consumer_scale"])
    best_params = FoodWebParams(
        **{
            **structural_kwargs,
            "h_fishing": 0.0,
            "pollution": 0.0,
            "gain_scale": best["gain_scale"],
        }
    )
    best_t, best_series = simulate_food_web(best_params, t_span, best_y0)
    best_metrics = q1_metrics(best_t, best_series)
    summary = {
        "consumer_scale": best["consumer_scale"],
        "gain_scale": best["gain_scale"],
        "loss": best["loss"],
        "metrics": best_metrics,
        "calibration_scope": "fit aggregate_carp_ratio_2025 only; reserve CPUE proxy for post-fit consistency",
        "gain_grid": [float(gain_grid[0]), float(gain_grid[-1]), int(len(gain_grid))],
    }
    return best_y0, best_params, summary


def evaluate_counterfactual_h0(
    base_params: FoodWebParams, y0: np.ndarray, t_span: tuple[float, float], ban_metrics: dict
) -> tuple[FoodWebParams, np.ndarray, dict]:
    h0 = Q1_COUNTERFACTUAL_H0
    params = FoodWebParams(**{**base_params.__dict__, "h_fishing": h0})
    t, series = simulate_food_web(params, t_span, y0)

```

```

metrics = q1_metrics(t, series)
max_carp_ratio = float(np.max(_normalize(total_carp(series))))
summary = {
    "h0": h0,
    "metrics": metrics,
    "selection_rule": "fixed mid-range counterfactual within [0.10, 0.40]; not estimated from observations",
    "aggregate_lower_than_ban": bool(metrics["aggregate_carp_ratio_2025"] < ban_metrics["
        aggregate_carp_ratio_2025"]),
    "cpue_lower_than_ban": bool(metrics["cpue_proxy_ratio_2025"] < ban_metrics["cpue_proxy_ratio_2025"]),
    "max_carp_ratio": max_carp_ratio,
    "no_overshoot": bool(max_carp_ratio <= 1.05),
}
return params, series, summary

def run_q1_counterfactual_scan(
    base_params: FoodWebParams,
    y0: np.ndarray,
    t_span: tuple[float, float],
    ban_metrics: dict,
) -> dict:
    samples = []
    for h0 in Q1_COUNTERFACTUAL_SCAN:
        params = FoodWebParams(**{**base_params.__dict__, "h_fishing": float(h0)})
        t, series = simulate_food_web(params, t_span, y0)
        metrics = q1_metrics(t, series)
        aggregate_gap = ban_metrics["aggregate_carp_ratio_2025"] - metrics["aggregate_carp_ratio_2025"]
        cpue_gap = ban_metrics["cpue_proxy_ratio_2025"] - metrics["cpue_proxy_ratio_2025"]
        max_carp_ratio = float(np.max(_normalize(total_carp(series))))
        samples.append(
            {
                "h0": float(h0),
                "aggregate_carp_ratio_2025": metrics["aggregate_carp_ratio_2025"],
                "cpue_proxy_ratio_2025": metrics["cpue_proxy_ratio_2025"],
                "aggregate_gap_vs_ban": float(aggregate_gap),
                "aggregate_gap_vs_ban_share": float(aggregate_gap / ban_metrics["aggregate_carp_ratio_2025"]),
                "cpue_gap_vs_ban": float(cpue_gap),
                "cpue_gap_vs_ban_share": float(cpue_gap / ban_metrics["cpue_proxy_ratio_2025"]),
                "ban_stronger_than_noban": bool(metrics["aggregate_carp_ratio_2025"] < ban_metrics["
                    aggregate_carp_ratio_2025"]),
                "no_overshoot": bool(max_carp_ratio <= 1.05),
            }
        )
    )

h0_ge_015 = [sample for sample in samples if sample["h0"] >= 0.15 - 1e-12]
return {
    "scan_range": [float(Q1_COUNTERFACTUAL_SCAN[0]), float(Q1_COUNTERFACTUAL_SCAN[-1])],
    "points": int(len(Q1_COUNTERFACTUAL_SCAN)),
    "samples": samples,
    "all_ban_stronger_than_noban": bool(all(sample["ban_stronger_than_noban"] for sample in samples)),
    "all_no_overshoot": bool(all(sample["no_overshoot"] for sample in samples)),
    "aggregate_gap_check_for_h0_ge_0_15": {
        "threshold_share": 0.30,
        "min_gap_share": float(min(sample["aggregate_gap_vs_ban_share"] for sample in h0_ge_015)),
        "all_above_threshold": bool(all(sample["aggregate_gap_vs_ban_share"] > 0.30 for sample in h0_ge_015)),
    },
}

def run_q1_sensitivity(
    base_y0_template: np.ndarray,
    fitted_y0: np.ndarray,

```

```

base_params: FoodWebParams,
h0: float,
t_span: tuple[float, float],
) -> dict:
    scenarios = {}
    settings = {
        "gain_scale": ("params", [0.9, 1.1]),
        "consumer_scale": ("y0", [0.9, 1.1]),
        "h0": ("h0", [0.9, 1.1]),
    }
    for name, (kind, scales) in settings.items():
        scenarios[name] = {}
        for scale in scales:
            if kind == "params":
                params = FoodWebParams(**{**base_params.__dict__, "gain_scale": base_params.gain_scale * scale})
                y0 = fitted_y0.copy()
                t, series = simulate_food_web(params, t_span, y0)
            elif kind == "y0":
                params = base_params
                y0 = fitted_y0.copy()
                y0[4:] *= scale
                t, series = simulate_food_web(params, t_span, y0)
            else:
                params = FoodWebParams(**{**base_params.__dict__, "h_fishing": h0 * scale})
                y0 = fitted_y0.copy()
                t, series = simulate_food_web(params, t_span, y0)
            metrics = q1_metrics(t, series)
            scenarios[name][f"{scale:.1f}x"] = {
                "aggregate_carp_ratio_2025": metrics["aggregate_carp_ratio_2025"],
                "cpue_proxy_ratio_2025": metrics["cpue_proxy_ratio_2025"],
                "pressure_end": metrics["pressure_end"],
                "water_min": metrics["water_min"],
                "water_min_year": metrics["water_min_year"],
            }
    }

structural_groups = {
    "resource_growth": ["rw", "rp", "rz", "rn"],
    "coupling_strength": ["a_wg", "a_ps", "a_zb", "a_nq"],
    "conversion_efficiency": ["e_wg", "e_ps", "e_zb", "e_nq"],
    "natural_loss": ["m_g", "m_s", "m_b", "m_q"],
}

structural_recalibration = {}
for group_name, field_names in structural_groups.items():
    structural_recalibration[group_name] = {}
    for scale in (0.8, 1.2):
        overrides = {field: getattr(base_params, field) * scale for field in field_names}
        y0_refit, params_refit, fit_summary = calibrate_q1(base_y0_template, t_span, structural_overrides=
            overrides)
        t_ban, ban_series = simulate_food_web(params_refit, t_span, y0_refit)
        ban_metrics = q1_metrics(t_ban, ban_series)
        noban_params = FoodWebParams(**{**params_refit.__dict__, "h_fishing": h0})
        _, noban_series = simulate_food_web(noban_params, t_span, y0_refit)
        noban_metrics = q1_metrics(t_ban, noban_series)
        structural_recalibration[group_name][f"{scale:.1f}x"] = {
            "refitted_gain_scale": fit_summary["gain_scale"],
            "aggregate_carp_ratio_2025": ban_metrics["aggregate_carp_ratio_2025"],
            "cpue_proxy_ratio_2025": ban_metrics["cpue_proxy_ratio_2025"],
            "pressure_end": ban_metrics["pressure_end"],
            "water_end": ban_metrics["water_end"],
            "water_min": ban_metrics["water_min"],
            "water_min_year": ban_metrics["water_min_year"],
            "ban_stronger_than_noban": bool(

```

```

        ban_metrics["aggregate_carp_ratio_2025"] > noban_metrics["aggregate_carp_ratio_2025"]
    ),
}
scenarios["structural_grouped_recalibration"] = {
    "perturbation": "groupwise +/-20% structural perturbation with aggregate-carp anchor re-fitted",
    "groups": structural_recalibration,
}
return scenarios

def run_q1_cpue_weight_sensitivity(t: np.ndarray, series: np.ndarray) -> dict:
    idx_2025 = int(np.argmax(np.abs(t - (Q1_TIME_ANCHORS["monitor_year"] - Q1_TIME_ANCHORS["policy_start"]))))
    scanned_weights = np.linspace(0.5, 1.0, 6)
    ratios = {}
    values = []
    for black_carp_weight in scanned_weights:
        weights = np.array([1.0, 1.0, 1.0, float(black_carp_weight)], dtype=float)
        cpue = cpue_proxy(series, weights=weights)
        ratio_2025 = float(_normalize(cpue)[idx_2025])
        ratios[f"{black_carp_weight:.1f}"] = ratio_2025
        values.append(ratio_2025)
    return {
        "black_carp_weight_range": [0.5, 1.0],
        "ratios_2025_by_weight": ratios,
        "min_ratio_2025": float(min(values)),
        "max_ratio_2025": float(max(values)),
    }

def build_q1_summary(
    t: np.ndarray,
    ban: np.ndarray,
    noban: np.ndarray,
    calibration: dict,
    h0_summary: dict,
    counterfactual_scan: dict,
    sensitivity: dict,
) -> dict:
    ban_metrics = q1_metrics(t, ban)
    noban_metrics = q1_metrics(t, noban)
    rel_errors = {
        "aggregate_carp_ratio_2025": float(
            abs(ban_metrics["aggregate_carp_ratio_2025"] - Q1_TARGETS["aggregate_carp_ratio_2025"]) / Q1_TARGETS["
            aggregate_carp_ratio_2025"]
        ),
        "cpue_proxy_ratio_2025": float(
            abs(ban_metrics["cpue_proxy_ratio_2025"] - Q1_TARGETS["cpue_proxy_ratio_2025"]) / Q1_TARGETS["
            cpue_proxy_ratio_2025"]
        ),
    }
    return {
        "time_anchor_years": Q1_TIME_ANCHORS,
        "targets": Q1_TARGETS,
        "fitted_parameters": {
            "consumer_scale": calibration["consumer_scale"],
            "gain_scale": calibration["gain_scale"],
            "counterfactual_h0": h0_summary["h0"],
            "cpue_weights": Q1_CPUE_WEIGHTS.tolist(),
            "calibration_loss": calibration["loss"],
            "calibration_scope": calibration["calibration_scope"],
        },
        "parameter_roles": {

```

```

        "consumer_scale": "fixed initial-state normalization; not estimated from 2025 observations",
        "gain_scale": "single fitted parameter for the 2025 aggregate-carp hard anchor",
        "counterfactual_h0": "fixed counterfactual fishing-pressure parameter; not part of observation fitting"
    },
    "cpue_weights": "heuristic proxy weights for local consistency checking only",
},
"counterfactual_check": {
    "selection_rule": h0_summary["selection_rule"],
    "aggregate_lower_than_ban": h0_summary["aggregate_lower_than_ban"],
    "cpue_lower_than_ban": h0_summary["cpue_lower_than_ban"],
    "max_carp_ratio": h0_summary["max_carp_ratio"],
    "no_overshoot": h0_summary["no_overshoot"],
},
"counterfactual_scan": counterfactual_scan,
"aggregate_carp_ratio_2025": {
    "target": Q1_TARGETS["aggregate_carp_ratio_2025"],
    "ban": ban_metrics["aggregate_carp_ratio_2025"],
    "noban": noban_metrics["aggregate_carp_ratio_2025"],
},
"cpue_proxy_ratio_2025": {
    "target": Q1_TARGETS["cpue_proxy_ratio_2025"],
    "ban": ban_metrics["cpue_proxy_ratio_2025"],
    "noban": noban_metrics["cpue_proxy_ratio_2025"],
},
"relative_errors": rel_errors,
"ban": ban_metrics,
"noban": noban_metrics,
"milestones": {
    "ban": ban_metrics["milestones"],
    "noban": noban_metrics["milestones"],
},
"sensitivity": sensitivity,
"cpue_weight_sensitivity": run_q1_cpue_weight_sensitivity(t, ban),
}

def plot_q1_comparison(t: np.ndarray, ban: np.ndarray, noban: np.ndarray, q1_summary: dict) -> dict:
    labels = ["W", "Ph", "Z", "N", "G", "S", "B", "Q"]
    nice = {
        "W": "Aquatic plants",
        "Ph": "Phytoplankton",
        "Z": "Zooplankton",
        "N": "Benthos",
        "G": "Grass carp",
        "S": "Silver carp",
        "B": "Bighead carp",
        "Q": "Black carp",
    }

    fig, axes = plt.subplots(2, 2, figsize=(13, 9), sharex=False)
    anchor_t_2022 = Q1_TIME_ANCHORS["gazette_year"] - Q1_TIME_ANCHORS["policy_start"]
    anchor_t_2025 = Q1_TIME_ANCHORS["monitor_year"] - Q1_TIME_ANCHORS["policy_start"]

    for idx, ax in enumerate([axes[0, 0], axes[0, 1]]):
        data = ban if idx == 0 else noban
        title = "Fishing ban" if idx == 0 else "No ban"
        for i, lab in enumerate(labels):
            ax.plot(t, _normalize(data[i]), label=nice[lab], lw=1.8)
            ax.axvline(anchor_t_2022, color="#666666", lw=1.0, ls="--")
            ax.axvline(anchor_t_2025, color="#d62728", lw=1.0, ls=":")
        ax.set_title(title)
        ax.set_ylabel("Normalized biomass")

```



```

ax.legend(ncol=4, fontsize=8, frameon=False)
ax.set_xlabel("Time since 2021 (years)")

ax_obs = axes[1, 0]
categories = ["Carp biomass ratio", "CPUE proxy ratio"]
observed = [
    q1_summary["targets"]["aggregate_carp_ratio_2025"],
    q1_summary["targets"]["cpue_proxy_ratio_2025"],
]
simulated = [
    q1_summary["aggregate_carp_ratio_2025"]["ban"],
    q1_summary["cpue_proxy_ratio_2025"]["ban"],
]
x = np.arange(len(categories))
width = 0.32
ax_obs.bar(x - width / 2, observed, width=width, label="Observed target", color="#4c78a8")
ax_obs.bar(x + width / 2, simulated, width=width, label="Simulated", color="#f58518")
ax_obs.set_xticks(x)
ax_obs.set_xticklabels(categories, rotation=8)
ax_obs.set_ylabel("Ratio to 2021 baseline")
ax_obs.set_title("2025 hard anchor and local consistency check")
ax_obs.legend(frameon=False)

ax_mech = axes[1, 1]
phi_ban = pressure_index(ban)
phi_noban = pressure_index(noban)
water_ban = _normalize(ban[0])
water_noban = _normalize(noban[0])
ax_mech.plot(t, phi_ban, label="Phi(t) - ban", lw=2.0, color="#d62728")
ax_mech.plot(t, phi_noban, label="Phi(t) - no ban", lw=1.8, ls="--", color="#ff9896")
ax_mech.plot(t, water_ban, label="Water plants - ban", lw=2.0, color="#2ca02c")
ax_mech.plot(t, water_noban, label="Water plants - no ban", lw=1.8, ls="--", color="#98df8a")
ax_mech.axvline(anchor_t_2025, color="#666666", lw=1.0, ls=":")
ax_mech.set_xlabel("Time since 2021 (years)")
ax_mech.set_ylabel("Index / normalized biomass")
ax_mech.set_title("Mechanism: pressure vs basal recovery")
ax_mech.legend(frameon=False, fontsize=8)

fig.suptitle("Question 1: calibrated trajectories and empirical anchors", y=0.99, fontsize=12)
fig.tight_layout()
_safe_savefig(fig, FIG_DIR / "fig01_q1_foodweb.png")
plt.close(fig)

return q1_summary

def plot_q2_comparison(t: np.ndarray, base: np.ndarray, polluted: np.ndarray) -> dict:
    labels = ["M", "D", "S", "V"]
    nice = {"M": "Forage fish", "D": "Porpoise", "S": "Sturgeon", "V": "Invader"}
    fig, ax = plt.subplots(figsize=(9, 4.8))
    for i, lab in enumerate(labels):
        ax.plot(t, _normalize(base[i]), lw=2.0, label=f"{nice[lab]} - baseline")
    for i, lab in enumerate(labels):
        ax.plot(t, _normalize(polluted[i]), lw=1.8, ls="--", label=f"{nice[lab]} - polluted")
    ax.set_xlabel("Time (years)")
    ax.set_ylabel("Normalized biomass")
    ax.set_title("Question 2 / 5: rare species and invasion module")
    ax.legend(ncol=2, fontsize=8, frameon=False)
    fig.tight_layout()
    _safe_savefig(fig, FIG_DIR / "fig02_q2_rare_species.png")
    plt.close(fig)

```

```

base_end = q2_end_values(base)
polluted_end = q2_end_values(polluted)
labels_for_ratio = ("M", "D", "A", "V")
polluted_vs_base = {
    lab: float(polluted_end[lab] / base_end[lab] - 1.0) if base_end[lab] else None for lab in labels_for_ratio
}
polluted_vs_base["S"] = polluted_vs_base["A"]

return {
    "base_end": base_end,
    "polluted_end": polluted_end,
    "polluted_vs_base_ratio": polluted_vs_base,
}

def summarize_q2_upstream_proxy(t: np.ndarray, series: np.ndarray, basal_weight: float = Q2_PROXY_BASAL_WEIGHT)
    -> dict:
    basal, carp = build_ecosystem_index_components(series)
    proxy = build_ecosystem_index(series, basal_weight=basal_weight)
    idx_2025 = int(np.argmax(np.abs(t - (Q1_TIME_ANCHORS["monitor_year"] - Q1_TIME_ANCHORS["policy_start"]))))
    return {
        "definition": "E(t)=w_basal*E_basal(t)+(1-w_basal)*E_carp(t)",
        "default_weights": {
            "basal": float(basal_weight),
            "carp": float(1.0 - basal_weight),
        },
        "component_construction": {
            "E_basal": "0.30*W + 0.25*Ph + 0.25*Z + 0.20*N after within-layer normalization",
            "E_carp": "0.28*G + 0.24*S + 0.24*B + 0.24*Q after within-layer normalization",
        },
        "calibrated_2025_levels": {
            "E_basal": float(basal[idx_2025]),
            "E_carp": float(carp[idx_2025]),
            "E_total": float(proxy[idx_2025]),
            "B_carp": float(_normalize(total_carp(series))[idx_2025]),
        },
        "trajectory_diagnostics": {
            "corr_with_B_carp": {
                "E_basal": float(np.corrcoef(basal, _normalize(total_carp(series)))[0, 1]),
                "E_carp": float(np.corrcoef(carp, _normalize(total_carp(series)))[0, 1]),
            },
            "tail_std": {
                "E_basal": float(np.std(basal)),
                "E_carp": float(np.std(carp)),
            },
        },
        "choice_note": (
            "The default blend keeps the resource-side signal explicit so that the q2 forcing proxy "
            "does not collapse into a pure fish-self-feedback index; robustness is checked by the weight scan below"
            "."
        ),
    }

def run_q2_sensitivity(
    rare_base_params: RareSpeciesParams,
    t_span: tuple[float, float],
    y0_q2_base: np.ndarray,
    food_supply_ban: Callable[[float], float],
) -> dict:
    sens_params = ["barrier", "release_s", "e_d", "e_s", "a_d", "a_s"]
    sens_scale = 0.20

```

```

sens_results: dict[str, dict[str, float]] = {}
_, base = simulate_rare_species(rare_base_params, t_span, y0_q2_base, food_supply_ban)
base_d = float(base[1, -1])
base_a = float(base[2, -1])
for pname in sens_params:
    base_val = getattr(rare_base_params, pname)
    row: dict[str, float] = {"base_value": float(base_val)}
    for direction, factor in [("plus20", 1 + sens_scale), ("minus20", 1 - sens_scale)]:
        perturbed = rare_base_params.__class__(
            **{k: (v * factor if k == pname else v) for k, v in rare_base_params.__dict__.items()}
        )
        _, sol_sens = simulate_rare_species(perturbed, t_span, y0_q2_base, food_supply_ban)
        row[f"D_{direction}"] = float(sol_sens[1, -1] / base_d - 1.0)
        row[f"A_{direction}"] = float(sol_sens[2, -1] / base_a - 1.0)
        row[f"S_{direction}"] = row[f"A_{direction}"]
    sens_results[pname] = row
return sens_results

def run_q2_proxy_weight_scan(
    t: np.ndarray,
    ban_series: np.ndarray,
    polluted_q1_series: np.ndarray,
    t_span: tuple[float, float],
    y0_q2_base: np.ndarray,
    scenario_params: dict[str, RareSpeciesParams],
) -> dict:
    def scenario_endpoints(basal_weight: float) -> dict[str, dict[str, float]]:
        food_supply_ban = interp1d(
            t,
            build_ecosystem_index(ban_series, basal_weight=basal_weight),
            kind="linear",
            fill_value="extrapolate",
            bounds_error=False,
        )
        food_supply_polluted = interp1d(
            t,
            build_ecosystem_index(polluted_q1_series, basal_weight=basal_weight),
            kind="linear",
            fill_value="extrapolate",
            bounds_error=False,
        )
        endpoints = {}
        for name, params in scenario_params.items():
            food_supply = food_supply_polluted if name == "polluted" else food_supply_ban
            _, sol = simulate_rare_species(params, t_span, y0_q2_base, food_supply)
            endpoints[name] = q2_end_values(sol)
        return endpoints

    default_endpoints = scenario_endpoints(Q2_PROXY_BASAL_WEIGHT)
    samples = []
    max_d_base = 0.0
    max_a_base = 0.0
    max_d_all = 0.0
    max_a_all = 0.0
    porpoise_orders = []
    sturgeon_orders = []

    for basal_weight in Q2_PROXY_WEIGHT_SCAN:
        endpoints = scenario_endpoints(float(basal_weight))
        porpoise_order = sorted(endpoints, key=lambda name: endpoints[name]["D"], reverse=True)
        sturgeon_order = sorted(endpoints, key=lambda name: endpoints[name]["A"], reverse=True)

```

```

porpoise_orders.append(porpoise_order)
sturgeon_orders.append(sturgeon_order)

local_d_all = 0.0
local_a_all = 0.0
for name in endpoints:
    local_d_all = max(local_d_all, abs(endpoints[name]["D"] / default_endpoints[name]["D"] - 1.0))
    local_a_all = max(local_a_all, abs(endpoints[name]["A"] / default_endpoints[name]["A"] - 1.0))
local_d_base = abs(endpoints["base"]["D"] / default_endpoints["base"]["D"] - 1.0)
local_a_base = abs(endpoints["base"]["A"] / default_endpoints["base"]["A"] - 1.0)
max_d_base = max(max_d_base, local_d_base)
max_a_base = max(max_a_base, local_a_base)
max_d_all = max(max_d_all, local_d_all)
max_a_all = max(max_a_all, local_a_all)

samples.append(
    {
        "basal_weight": float(basal_weight),
        "carp_weight": float(1.0 - basal_weight),
        "scenario_endpoints": {
            name: {"D": endpoints[name]["D"], "A": endpoints[name]["A"]} for name in endpoints
        },
        "porpoise_order": porpoise_order,
        "sturgeon_order": sturgeon_order,
        "max_abs_relative_change_base": {
            "D": float(local_d_base),
            "A": float(local_a_base),
        },
        "max_abs_relative_change_all_scenarios": {
            "D": float(local_d_all),
            "A": float(local_a_all),
        },
    }
)

return {
    "scan_range": [float(Q2_PROXY_WEIGHT_SCAN[0]), float(Q2_PROXY_WEIGHT_SCAN[-1])],
    "default_weights": {
        "basal": float(Q2_PROXY_BASAL_WEIGHT),
        "carp": float(1.0 - Q2_PROXY_BASAL_WEIGHT),
    },
    "samples": samples,
    "max_abs_relative_change_base": {
        "D": float(max_d_base),
        "A": float(max_a_base),
    },
    "max_abs_relative_change_all_scenarios": {
        "D": float(max_d_all),
        "A": float(max_a_all),
    },
    "stability_checks": {
        "porpoise_order_preserved": bool(all(order == porpoise_orders[0] for order in porpoise_orders)),
        "sturgeon_order_preserved": bool(all(order == sturgeon_orders[0] for order in sturgeon_orders)),
        "barrier_penalty_preserved": bool(
            all(
                sample["scenario_endpoints"]["barrier"]["D"] < sample["scenario_endpoints"]["base"]["D"]
                and sample["scenario_endpoints"]["barrier"]["A"] < sample["scenario_endpoints"]["base"]["A"]
                for sample in samples
            )
        ),
        "release_lift_preserved": bool(
            all(

```

```

        sample["scenario_endpoints"]["base"]["A"] > sample["scenario_endpoints"]["no_release"]["A"]
    for sample in samples
)
),
"pollution_penalty_preserved": bool(
    all(
        sample["scenario_endpoints"]["polluted"]["D"] < sample["scenario_endpoints"]["base"]["D"]
        and sample["scenario_endpoints"]["polluted"]["A"] < sample["scenario_endpoints"]["base"]["A"]
        for sample in samples
    )
),
},
}

def run_q2_barrier_multiplier_scan(
    t: np.ndarray,
    t_span: tuple[float, float],
    y0_q2_base: np.ndarray,
    scenario_params: dict[str, RareSpeciesParams],
    food_supply_ban: Callable[[float], float],
    food_supply_polluted: Callable[[float], float],
) -> dict:
    def scenario_endpoints(multiplier: float) -> dict[str, dict[str, float]]:
        endpoints = {}
        for name, params in scenario_params.items():
            food_supply = food_supply_polluted if name == "polluted" else food_supply_ban
            params_now = RareSpeciesParams(**{**params.__dict__, "sturgeon_barrier_multiplier": float(multiplier)})
            _, sol = simulate_rare_species(params_now, t_span, y0_q2_base, food_supply)
            endpoints[name] = q2_end_values(sol)
        return endpoints

    default_endpoints = scenario_endpoints(Q2_STURGEON_BARRIER_MULTIPLIER)
    samples = []
    max_d_base = 0.0
    max_a_base = 0.0
    max_d_all = 0.0
    max_a_all = 0.0
    porpoise_orders = []
    sturgeon_orders = []

    for multiplier in Q2_STURGEON_BARRIER_SCAN:
        endpoints = scenario_endpoints(float(multiplier))
        porpoise_order = sorted(endpoints, key=lambda name: endpoints[name]["D"], reverse=True)
        sturgeon_order = sorted(endpoints, key=lambda name: endpoints[name]["A"], reverse=True)
        porpoise_orders.append(porpoise_order)
        sturgeon_orders.append(sturgeon_order)

    local_d_all = 0.0
    local_a_all = 0.0
    for name in endpoints:
        local_d_all = max(local_d_all, abs(endpoints[name]["D"] / default_endpoints[name]["D"] - 1.0))
        local_a_all = max(local_a_all, abs(endpoints[name]["A"] / default_endpoints[name]["A"] - 1.0))
    local_d_base = abs(endpoints["base"]["D"] / default_endpoints["base"]["D"] - 1.0)
    local_a_base = abs(endpoints["base"]["A"] / default_endpoints["base"]["A"] - 1.0)
    max_d_base = max(max_d_base, local_d_base)
    max_a_base = max(max_a_base, local_a_base)
    max_d_all = max(max_d_all, local_d_all)
    max_a_all = max(max_a_all, local_a_all)

    samples.append(
        {

```

```

        "sturgeon_barrier_multiplier": float(multiplier),
        "scenario_endpoints": {
            name: {"D": endpoints[name]["D"], "A": endpoints[name]["A"]} for name in endpoints
        },
        "porpoise_order": porpoise_order,
        "sturgeon_order": sturgeon_order,
        "max_abs_relative_change_base": {
            "D": float(local_d_base),
            "A": float(local_a_base),
        },
        "max_abs_relative_change_all_scenarios": {
            "D": float(local_d_all),
            "A": float(local_a_all),
        },
    }
}

)

return {
    "scan_range": [float(Q2_STURGEON_BARRIER_SCAN[0]), float(Q2_STURGEON_BARRIER_SCAN[-1])],
    "default_multiplier": float(Q2_STURGEON_BARRIER_MULTIPLIER),
    "samples": samples,
    "max_abs_relative_change_base": {
        "D": float(max_d_base),
        "A": float(max_a_base),
    },
    "max_abs_relative_change_all_scenarios": {
        "D": float(max_d_all),
        "A": float(max_a_all),
    },
    "stability_checks": {
        "porpoise_order_preserved": bool(all(order == porpoise_orders[0] for order in porpoise_orders)),
        "sturgeon_order_preserved": bool(all(order == sturgeon_orders[0] for order in sturgeon_orders)),
        "barrier_penalty_preserved": bool(
            all(
                sample["scenario_endpoints"]["barrier"]["D"] < sample["scenario_endpoints"]["base"]["D"]
                and sample["scenario_endpoints"]["barrier"]["A"] < sample["scenario_endpoints"]["base"]["A"]
                for sample in samples
            )
        ),
        "release_lift_preserved": bool(
            all(
                sample["scenario_endpoints"]["base"]["A"] > sample["scenario_endpoints"]["no_release"]["A"]
                for sample in samples
            )
        ),
        "pollution_penalty_preserved": bool(
            all(
                sample["scenario_endpoints"]["polluted"]["D"] < sample["scenario_endpoints"]["base"]["D"]
                and sample["scenario_endpoints"]["polluted"]["A"] < sample["scenario_endpoints"]["base"]["A"]
                for sample in samples
            )
        ),
    },
}

}

def compute_q2_directional_validation(t: np.ndarray, base: np.ndarray) -> dict:
    idx = int(np.argmax(np.abs(t - Q2_DIRECTIONAL_VALIDATION_T)))
    a_ratio = float(base[2, idx] / max(base[2, 0], 1e-12))
    d_ratio = float(base[1, idx] / max(base[1, 0], 1e-12))
    return {
        "model_time": float(Q2_DIRECTIONAL_VALIDATION_T),

```

```

        "calendar_year": int(Q1_TIME_ANCHORS["policy_start"] + Q2_DIRECTIONAL_VALIDATION_T),
        "A_ratio": a_ratio,
        "D_ratio": d_ratio,
        "sturgeon_positive_growth": bool(a_ratio > 1.0),
        "porpoise_positive_growth": bool(d_ratio > 1.0),
    }

def _positive_runs(ks: np.ndarray, values: np.ndarray, min_len: int = 3) -> list[tuple[int, int]]:
    mask = values > 0
    runs = []
    start = None
    for i, flag in enumerate(mask):
        if flag and start is None:
            start = i
        if start is not None and (not flag or i == len(mask) - 1):
            end = i if flag and i == len(mask) - 1 else i - 1
            if end - start + 1 >= min_len:
                runs.append((start, end))
            start = None
    return runs

def _peak_times(t: np.ndarray, x: np.ndarray) -> np.ndarray:
    if len(x) < 3:
        return np.array([], dtype=float)
    mid = x[1:-1]
    peak_mask = (mid > x[:-2]) & (mid >= x[2:])
    peak_idx = np.where(peak_mask)[0] + 1
    return t[peak_idx]

def _orbit_descriptors(sol, window_start: float = 90.0) -> dict:
    mask = sol.t >= window_start
    if not np.any(mask):
        mask = np.ones_like(sol.t, dtype=bool)

    t_tail = sol.t[mask]
    x1_tail = sol.y[0][mask]
    peak_times = _peak_times(t_tail, x1_tail)
    peak_intervals = np.diff(peak_times) if len(peak_times) >= 2 else np.array([], dtype=float)
    mean_peak_interval = float(np.mean(peak_intervals)) if len(peak_intervals) else None
    peak_interval_cv = float(np.std(peak_intervals) / np.mean(peak_intervals)) if len(peak_intervals) and np.mean(
        peak_intervals) > 0 else None

    return {
        "window_start": float(t_tail[0]),
        "window_end": float(t_tail[-1]),
        "tail_mean_x1": float(np.mean(x1_tail)),
        "tail_std_x1": float(np.std(x1_tail)),
        "x1_min": float(np.min(x1_tail)),
        "x1_max": float(np.max(x1_tail)),
        "x1_peak_to_peak": float(np.ptp(x1_tail)),
        "peak_count": int(len(peak_times)),
        "mean_peak_interval": mean_peak_interval,
        "peak_interval_cv": peak_interval_cv,
    }

def _scan_window_summary(ks: np.ndarray, mles: np.ndarray, scan_min: float = 3.0, scan_max: float = 3.4) -> dict:
    mask = (ks >= scan_min) & (ks <= scan_max)
    scan_ks = ks[mask]

```

```

scan_mles = mles[mask]
runs = _positive_runs(scan_ks, scan_mles, min_len=3)
continuous_window = None
if runs:
    start_idx, end_idx = runs[0]
    continuous_window = {
        "start": float(scan_ks[start_idx]),
        "end": float(scan_ks[end_idx]),
        "step": float(scan_ks[1] - scan_ks[0]) if len(scan_ks) > 1 else 0.0,
    }
positive_idx = np.where(scan_mles > 0)[0]
return {
    "scan_range": [float(scan_min), float(scan_max)],
    "continuous_positive_window": continuous_window,
    "first_positive_k": float(scan_ks[positive_idx[0]]) if len(positive_idx) else None,
    "positive_point_count": int(len(positive_idx)),
    "mle_at_3.20": float(scan_mles[int(np.argmin(np.abs(scan_ks - 3.20)))]),
    "mle_at_3.22": float(scan_mles[int(np.argmin(np.abs(scan_ks - 3.22)))]),
}

def plot_hp_results():
    coarse_ks = np.linspace(1.4, 4.8, 44)
    fine_ks = np.linspace(3.0, 3.4, 41)
    ks = np.unique(np.concatenate([coarse_ks, fine_ks]))
    y0 = np.array([0.8, 0.2, 0.2], dtype=float)
    scan_totals = [90.0, 150.0, 200.0]
    numerical_settings = {
        "dt": 0.01,
        "transient_time": 40.0,
        "total_time": scan_totals[0],
        "initial_state": y0.tolist(),
    }

    snapshots = {}
    mle_scans = {}

    for total_time in scan_totals:
        window_values = []
        for k in ks:
            p = HPPParams(k=float(k))
            window_values.append(
                lyapunov_max(
                    p,
                    y0=y0,
                    dt=numerical_settings["dt"],
                    t_trans=numerical_settings["transient_time"],
                    t_total=total_time,
                )
            )
        if total_time == scan_totals[0] and k not in snapshots:
            t = np.linspace(0, 180, 2500)
            sol = solve_ivp(lambda tt, yy: hp_rhs(tt, yy, p), (t[0], t[-1]), y0, t_eval=t, rtol=1e-7, atol=1e-9)
            snapshots[k] = sol
        mle_scans[total_time] = np.array(window_values, dtype=float)

    mles = mle_scans[scan_totals[0]]
    baseline_scan = _scan_window_summary(ks, mles)
    chaos_window = baseline_scan["continuous_positive_window"] or {
        "start": 3.22,
        "end": 3.22,
        "step": 0.0,
    }
}

```



```

chaos_candidates = np.where((ks >= chaos_window["start"]) & (mles > 0))[0]
if len(chaos_candidates):
    irregular_k = float(ks[chaos_candidates[np.argmax(mles[chaos_candidates])]])
else:
    irregular_k = 4.8

stable_k = 1.8
periodic_k = 2.7
irregular_k = float(irregular_k if irregular_k >= 3.0 else 4.8)
stable_obs_k = min(ks, key=lambda x: abs(x - stable_k))
periodic_obs_k = min(ks, key=lambda x: abs(x - periodic_k))
irregular_obs_k = min(ks, key=lambda x: abs(x - irregular_k))
stable = snapshots[stable_obs_k]
periodic = snapshots[periodic_obs_k]
irregular = snapshots[irregular_obs_k]

def nearest_summary(target_k: float) -> dict:
    idx = int(np.argmin(np.abs(ks - target_k)))
    snapshot = snapshots[float(ks[idx])]
    descriptors = _orbit_descriptors(snapshot, window_start=90.0)
    return {
        "target_k": float(target_k),
        "observed_k": float(ks[idx]),
        "mle_t90": float(mles[idx]),
        "trajectory_window": {
            "start": descriptors["window_start"],
            "end": descriptors["window_end"],
        },
        "descriptors": {
            "tail_mean_x1": descriptors["tail_mean_x1"],
            "tail_std_x1": descriptors["tail_std_x1"],
            "x1_min": descriptors["x1_min"],
            "x1_max": descriptors["x1_max"],
            "x1_peak_to_peak": descriptors["x1_peak_to_peak"],
            "peak_count": descriptors["peak_count"],
            "mean_peak_interval": descriptors["mean_peak_interval"],
            "peak_interval_cv": descriptors["peak_interval_cv"],
        },
    }

isolated_positive_candidates = [float(k) for k, mle in zip(ks, mles) if k < chaos_window["start"] and mle > 0]

def mle_at(k: float, dt: float = 0.01, t_total: float = 90.0, y0_override: np.ndarray | None = None) -> float:
    y0_local = y0 if y0_override is None else np.array(y0_override, dtype=float)
    return float(
        lyapunov_max(
            HPParams(k=float(k)),
            y0=y0_local,
            dt=dt,
            t_trans=numerical_settings["transient_time"],
            t_total=t_total,
        )
    )

local_bracket_checks = {
    "base_dt_0.010": {
        "dt": 0.010,
        "initial_state": y0.tolist(),
        "k_left": 3.20,
        "mle_left": mle_at(3.20, dt=0.010),
        "k_right": 3.22,
    }
}

```

```

        "mle_right": mle_at(3.22, dt=0.010),
    },
    "dt_0.008": {
        "dt": 0.008,
        "initial_state": y0.tolist(),
        "k_left": 3.20,
        "mle_left": mle_at(3.20, dt=0.008),
        "k_right": 3.22,
        "mle_right": mle_at(3.22, dt=0.008),
    },
    "dt_0.012": {
        "dt": 0.012,
        "initial_state": y0.tolist(),
        "k_left": 3.20,
        "mle_left": mle_at(3.20, dt=0.012),
        "k_right": 3.22,
        "mle_right": mle_at(3.22, dt=0.012),
    },
    "y0_perturbed": {
        "dt": 0.010,
        "initial_state": [0.8005, 0.1995, 0.2],
        "k_left": 3.20,
        "mle_left": mle_at(3.20, dt=0.010, y0_override=np.array([0.8005, 0.1995, 0.2], dtype=float)),
        "k_right": 3.22,
        "mle_right": mle_at(3.22, dt=0.010, y0_override=np.array([0.8005, 0.1995, 0.2], dtype=float)),
    },
}
bracket_sign_stable = all(
    item["mle_left"] < 0 and item["mle_right"] > 0 for item in local_bracket_checks.values()
)

fig_q3, axes_q3 = plt.subplots(1, 3, figsize=(13.2, 3.9), sharey=False)
for ax, sol, observed_k, label, color in [
    (axes_q3[0], stable, stable_obs_k, "Bounded oscillation", "#1f77b4"),
    (axes_q3[1], periodic, periodic_obs_k, "Periodic oscillation", "#2ca02c"),
    (axes_q3[2], irregular, irregular_obs_k, "Irregular sample", "#d62728"),
]:
    desc = _orbit_descriptors(sol, window_start=90.0)
    ax.plot(sol.t, sol.y[0], lw=1.3, color=color)
    ax.set_xlabel("Time")
    ax.set_title(
        f"{label}\nK={float(observed_k):.3f}, "
        f"$\\sigma(X_1)$={desc['tail_std_x1']:.3f}, "
        f"$CV_T$={0.0 if desc['peak_interval_cv'] is None else desc['peak_interval_cv']:.3f}"
    )
axes_q3[0].set_ylabel("X1")
fig_q3.tight_layout()
_safe_savefig(fig_q3, FIG_DIR / "fig03_q3_hastings_powell.png")
plt.close(fig_q3)

fig_q4, axes_q4 = plt.subplots(2, 1, figsize=(10.5, 8.2), sharex=False)
colors = {90.0: "#1f77b4", 150.0: "#ff7f0e", 200.0: "#2ca02c"}
for total_time in scan_totals:
    label = f"$t_{{total}}$={int(total_time)}"
    axes_q4[0].plot(ks, mle_scans[total_time], lw=1.5, color=colors[total_time], label=label)
    zoom_mask = (ks >= 3.0) & (ks <= 3.4)
    axes_q4[1].plot(ks[zoom_mask], mle_scans[total_time][zoom_mask], lw=1.5, color=colors[total_time], label=
        label)
for ax in axes_q4:
    ax.axhline(0.0, color="black", lw=1.0, ls="--")
axes_q4[1].axvline(3.20, color="#555555", lw=1.0, ls=":")
axes_q4[1].axvline(3.22, color="#999999", lw=1.0, ls=":")

```

```

axes_q4[0].set_ylabel("Largest Lyapunov exponent")
axes_q4[0].set_title("Finite-time MLE scan under different integration windows")
axes_q4[0].legend(frameon=False, ncol=3)
axes_q4[1].set_xlabel("K")
axes_q4[1].set_ylabel("Largest Lyapunov exponent")
axes_q4[1].set_title("Zoom near the short-window sign-crossing bracket")
fig_q4.tight_layout()
_safe_savefig(fig_q4, FIG_DIR / "fig04_q4_lyapunov_scan.png")
plt.close(fig_q4)

window_sensitivity_scan = {
    f"t_total_{int(total_time)}": _scan_window_summary(ks, mle_scans[total_time]) for total_time in scan_totals
}

return {
    "control_parameter": {
        "name": "K",
        "interpretation": "dimensionless effective carrying capacity",
    },
    "observation_window": {
        "start": 90.0,
        "end": 180.0,
        "observable": "X1 tail-window descriptors for q3",
    },
    "numerical_settings": numerical_settings,
    "ks": ks.tolist(),
    "mles": mles.tolist(),
    "chaos_window": chaos_window,
    "window_sensitivity_scan": window_sensitivity_scan,
    "local_bracket": {
        "left_k": 3.20,
        "right_k": 3.22,
        "criterion": "lambda(left)<0<lambda(right) under nearby numerical settings",
        "sign_stable": bracket_sign_stable,
    },
    "local_bracket_checks": local_bracket_checks,
    "irregular_rep_k": irregular_k,
    "representative_behaviors": {
        "bounded_oscillation": nearest_summary(stable_k),
        "periodic_oscillation": nearest_summary(periodic_k),
        "irregular_trajectory": nearest_summary(irregular_k),
    },
    "isolated_positive_candidates_before_window": isolated_positive_candidates,
    "diagnosis_rule": "finite-time MLE scan with a short-window baseline and cross-window drift check",
    "scope_note": (
        "finite-time MLE depends on integration-window choice; "
        "the short-window sign-crossing near 3.20--3.22 is local, while longer windows shift the first-positive region"
    ),
}

def extended_q4_robustness_test() -> dict:
    """Broader stress test for the local Q4 chaos-candidate bracket.

    The paper's main claim is intentionally local: under the baseline scan
    settings and nearby perturbations, K=3.20 stays negative while K=3.22
    turns positive. This helper widens the test grid so the paper can report
    how far that sign-crossing survives once initial states, step sizes, and
    integration windows are pushed farther away from the baseline setup.
    """
    y0_variants = [

```

```

    ("baseline", np.array([0.80, 0.20, 0.20], dtype=float)),
    ("x1_down", np.array([0.75, 0.25, 0.20], dtype=float)),
    ("x1_up", np.array([0.85, 0.15, 0.20], dtype=float)),
    ("x3_up", np.array([0.80, 0.20, 0.25], dtype=float)),
    ("x3_down", np.array([0.80, 0.20, 0.15], dtype=float)),
]
dts = [0.005, 0.008, 0.010, 0.012, 0.015]
t_totals = [90.0, 150.0, 200.0]
t_trans = 40.0
k_left = 3.20
k_right = 3.22

def sign_outcome(mle_left: float, mle_right: float) -> str:
    if mle_left < 0 < mle_right:
        return "sign_cross"
    if mle_left >= 0 and mle_right > 0:
        return "both_positive"
    if mle_left < 0 and mle_right <= 0:
        return "both_negative"
    return "reversed_or_zero_tied"

def aggregate(subset: list[dict]) -> dict:
    total = len(subset)
    passed = sum(1 for item in subset if item["sign_cross_detected"])
    return {
        "total": total,
        "passed": passed,
        "failed": total - passed,
        "pass_rate": float(passed / total) if total else 0.0,
    }

test_results = []
for y0_name, y0 in y0_variants:
    for dt in dts:
        for t_total in t_totals:
            mle_left = lyapunov_max(
                HPParams(k=k_left),
                y0=y0,
                dt=dt,
                t_trans=t_trans,
                t_total=t_total,
            )
            mle_right = lyapunov_max(
                HPParams(k=k_right),
                y0=y0,
                dt=dt,
                t_trans=t_trans,
                t_total=t_total,
            )
            outcome = sign_outcome(float(mle_left), float(mle_right))
            test_results.append(
                {
                    "y0_variant": y0_name,
                    "y0": y0.tolist(),
                    "dt": float(dt),
                    "t_total": float(t_total),
                    "t_trans": float(t_trans),
                    "k_left": float(k_left),
                    "k_right": float(k_right),
                    "mle_left": float(mle_left),
                    "mle_right": float(mle_right),
                    "sign_cross_detected": bool(outcome == "sign_cross"),
                }
            )

```

```

        "outcome": outcome,
    }
)

total_tests = len(test_results)
overall = aggregate(test_results)

by_y0 = {}
for y0_name, _ in y0_variants:
    subset = [item for item in test_results if item["y0_variant"] == y0_name]
    by_y0[y0_name] = aggregate(subset)

by_dt = {}
for dt in dts:
    subset = [item for item in test_results if item["dt"] == dt]
    by_dt[f"dt_{dt:.3f}"] = aggregate(subset)

by_t_total = {}
for t_total in t_totals:
    subset = [item for item in test_results if item["t_total"] == t_total]
    by_t_total[f"t_total_{int(t_total)}"] = aggregate(subset)

outcome_counts = {}
for label in ("sign_cross", "both_positive", "both_negative", "reversed_or_zero_tied"):
    outcome_counts[label] = sum(1 for item in test_results if item["outcome"] == label)

sign_cross_cases = [
    {
        "y0_variant": item["y0_variant"],
        "dt": item["dt"],
        "t_total": item["t_total"],
        "mle_left": item["mle_left"],
        "mle_right": item["mle_right"],
    }
    for item in test_results
    if item["sign_cross_detected"]
]

failure_examples = [
    {
        "y0_variant": item["y0_variant"],
        "dt": item["dt"],
        "t_total": item["t_total"],
        "mle_left": item["mle_left"],
        "mle_right": item["mle_right"],
        "outcome": item["outcome"],
    }
    for item in test_results
    if not item["sign_cross_detected"]
][:12]

return {
    "test_configuration": {
        "k_left": float(k_left),
        "k_right": float(k_right),
        "t_trans": float(t_trans),
        "y0_variants": {name: values.tolist() for name, values in y0_variants},
        "dt_values": dts,
        "t_total_values": t_totals,
        "total_combinations": total_tests,
    },
    "overall_statistics": overall,
    "by_initial_condition": by_y0,
}

```

```

        "by_step_size": by_dt,
        "by_integration_time": by_t_total,
        "outcome_counts": outcome_counts,
        "sign_cross_cases": sign_cross_cases,
        "failure_examples": failure_examples,
        "detailed_results": test_results,
        "interpretation": {
            "criterion": "MLE(K=3.20) < 0 < MLE(K=3.22) indicates a local sign-crossing bracket",
            "main_reading": (
                f"Sign crossing survives in {overall['passed']}/{total_tests} stress-test settings; "
                "the bracket should therefore be read as a local finite-time candidate rather than a universal "
                "threshold."
            ),
            "scope": "Broader stress test on the finite-time MLE bracket; results complement but do not replace the "
                    "paper's baseline local check.",
        },
    },
}

def plot_q5_scenarios(scenarios: dict[str, np.ndarray]) -> dict:
    names = list(scenarios.keys())
    metrics = ["Food", "Porpoise", "Sturgeon", "Invader", "Health"]
    tail_window = 200

    def tail_mean(series: np.ndarray) -> float:
        return float(np.mean(_normalize(series)[-tail_window:]))

    raw_indicator_matrix = []
    end_indicator_matrix = []
    for name in names:
        _, data = scenarios[name]
        m, d, s, v = data
        raw_indicator_matrix.append([tail_mean(m), tail_mean(d), tail_mean(s), tail_mean(v)])
        end_indicator_matrix.append(
            [float(_normalize(m)[-1]), float(_normalize(d)[-1]), float(_normalize(s)[-1]), float(_normalize(v)[-1])]
        )

    raw_indicator_matrix = np.array(raw_indicator_matrix, dtype=float)
    end_indicator_matrix = np.array(end_indicator_matrix, dtype=float)
    benefit_mask = np.array([True, True, True, False], dtype=bool)
    score_matrix = _normalize_indicator_matrix(raw_indicator_matrix, benefit_mask)
    entropy = entropy_weights(score_matrix)
    critic = critic_weights(score_matrix)
    equal = np.full(4, 0.25)
    combined = _positive_normalize(0.5 * (entropy + critic))

    method_weights = {
        "entropy": entropy,
        "critic": critic,
        "combined": combined,
        "equal": equal,
    }

    method_scores = {label: score_matrix @ weights for label, weights in method_weights.items()}
    base_scores = method_scores["combined"]
    matrix = np.column_stack([raw_indicator_matrix, base_scores])

    def ranking_from_scores(scores: np.ndarray) -> list[str]:
        return sorted(names, key=lambda name: scores[names.index(name)], reverse=True)

    base_order = ranking_from_scores(base_scores)
    method_rankings = {label: ranking_from_scores(scores) for label, scores in method_scores.items()}

```

```

rng = np.random.default_rng(20260515)
perturbation_samples = 2000
perturbation_orders = []
invasion_worst_count = 0
full_order_count = 0
for _ in range(perturbation_samples):
    local_weights = _positive_normalize(combined * rng.uniform(0.85, 1.15, size=combined.shape[0]))
    local_scores = score_matrix @ local_weights
    local_order = ranking_from_scores(local_scores)
    perturbation_orders.append(local_order)
    if local_order == base_order:
        full_order_count += 1
    if local_order[-1] == "Ban + pollution + invasion":
        invasion_worst_count += 1

ranking_stable = all(order == base_order for order in method_rankings.values()) and full_order_count ==
    perturbation_samples
invasion_worst_stable = all(order[-1] == "Ban + pollution + invasion" for order in method_rankings.values())
    and (
        invasion_worst_count == perturbation_samples
    )

fig, ax = plt.subplots(figsize=(10.4, 4.8))
x = np.arange(len(metrics))
width = 0.22
for i, name in enumerate(names):
    ax.bar(x + (i - (len(names) - 1) / 2) * width, matrix[i], width, label=name)
ax.set_xticks(x)
ax.set_xticklabels(metrics)
ax.set_ylim(0, max(1.05, float(matrix.max()) * 1.15))
ax.set_ylabel("Tail-mean level / score")
ax.set_title("Question 5: internal scenario comparison")
ax.legend(frameon=False)
fig.tight_layout()
_safe_savefig(fig, FIG_DIR / "fig05_q5_scenarios.png")
plt.close(fig)

result = {name: {m: float(matrix[i, j]) for j, m in enumerate(metrics)} for i, name in enumerate(names)}
ban_row = result["Ban only"]
polluted_row = result["Ban + pollution"]
invasion_row = result["Ban + pollution + invasion"]

def rel_change(row_now: dict[str, float], row_ref: dict[str, float]) -> dict[str, float | None]:
    out = {}
    for key in metrics:
        ref = row_ref[key]
        out[key] = float(row_now[key] / ref - 1.0) if ref else None
    return out

result["relative_changes"] = {
    "pollution_vs_ban_only": rel_change(polluted_row, ban_row),
    "invasion_vs_pollution": rel_change(invasion_row, polluted_row),
    "invasion_vs_ban_only": rel_change(invasion_row, ban_row),
}

result["end_values"] = {
    name: {
        "Food": float(end_indicator_matrix[i, 0]),
        "Porpoise": float(end_indicator_matrix[i, 1]),
        "Sturgeon": float(end_indicator_matrix[i, 2]),
        "Invader": float(end_indicator_matrix[i, 3]),
    }
}

```

```

        for i, name in enumerate(names)
    }
    result["robust_conclusions"] = {
        "full_ranking_stable": ranking_stable,
        "invasion_worst_stable": invasion_worst_stable,
        "base_vs_pollution_order_sensitive": not ranking_stable,
        "conservative_statement": (
            "Under entropy, CRITIC, equal, combined, and local renormalized perturbation weights, "
            "the ordering Ban only > Ban + pollution > Ban + pollution + invasion is preserved."
        ),
    }
}
result["objective_weighting"] = {
    "tail_window_steps": tail_window,
    "indicator_type": {
        "Food": "benefit",
        "Porpoise": "benefit",
        "Sturgeon": "benefit",
        "Invader": "cost",
    },
    "raw_tail_means": {
        name: {
            "Food": float(raw_indicator_matrix[i, 0]),
            "Porpoise": float(raw_indicator_matrix[i, 1]),
            "Sturgeon": float(raw_indicator_matrix[i, 2]),
            "Invader": float(raw_indicator_matrix[i, 3]),
        }
        for i, name in enumerate(names)
    },
    "positive_oriented_scores": {
        name: {
            "Food": float(score_matrix[i, 0]),
            "Porpoise": float(score_matrix[i, 1]),
            "Sturgeon": float(score_matrix[i, 2]),
            "Invader": float(score_matrix[i, 3]),
        }
        for i, name in enumerate(names)
    },
    "weights": {label: [float(x) for x in weights] for label, weights in method_weights.items()},
    "method_rankings": method_rankings,
    "perturbation_test": {
        "seed": 20260515,
        "samples": perturbation_samples,
        "factor_range": [0.85, 1.15],
        "full_ranking_stability_rate": float(full_order_count / perturbation_samples),
        "invasion_worst_stability_rate": float(invasion_worst_count / perturbation_samples),
    },
}
result["health_window_steps"] = tail_window
result["health_definition"] = (
    "combined entropy-CRITIC score of the positive-oriented tail-mean indicators; "
    "all weights are nonnegative and sum to 1"
)
result["scope_note"] = "internal scenario comparison under q2-normalized state variables; not an external ecological-health validation"
return result

def main():
    existing_results: dict[str, dict] = {}
    existing_summary_path = OUT_DIR / "summary.json"
    if existing_summary_path.exists():
        try:

```



```

        with open(existing_summary_path, "r", encoding="utf-8") as f:
            existing_results = json.load(f)
        except (json.JSONDecodeError, OSError):
            existing_results = {}

# Question 1: ban vs no-ban
y0_q1_base = np.array([0.88, 0.82, 0.76, 0.70, 0.46, 0.42, 0.38, 0.34], dtype=float)
t_q1 = (0.0, 10.0)
y0_q1, ban_params, calibration = calibrate_q1(y0_q1_base, t_q1)
t, ban = simulate_food_web(ban_params, t_q1, y0_q1)
ban_metrics = q1_metrics(t, ban)
noban_params, noban, h0_summary = evaluate_counterfactual_h0(ban_params, y0_q1, t_q1, ban_metrics)
counterfactual_scan = run_q1_counterfactual_scan(ban_params, y0_q1, t_q1, ban_metrics)
sensitivity = run_q1_sensitivity(y0_q1_base, y0_q1, ban_params, h0_summary["h0"], t_q1)
q1_summary = build_q1_summary(t, ban, noban, calibration, h0_summary, counterfactual_scan, sensitivity)
q1_summary = plot_q1_comparison(t, ban, noban, q1_summary)

# Question 2 / 5: rare species module driven by food supply from Q1
food_supply_ban = interp1d(t, build_ecosystem_index(ban), kind="linear", fill_value="extrapolate",
    bounds_error=False)

y0_q2_base = np.array([0.62, 1.00, 0.14, 0.02], dtype=float)
y0_q2_polluted = np.array([0.62, 1.00, 0.14, 0.02], dtype=float)
rare_base_params = RareSpeciesParams(barrier=0.015, pollution=0.0, release_s=0.012, e_d=0.44, m_d=0.045)
rare_barrier_params = RareSpeciesParams(barrier=0.05, pollution=0.0, release_s=0.012, e_d=0.44, m_d=0.045)
rare_polluted_params = RareSpeciesParams(barrier=0.05, pollution=0.28, release_s=0.012, e_d=0.40, m_d=0.050)

t2, base = simulate_rare_species(rare_base_params, t_q1, y0_q2_base, food_supply_ban)
_, barrier_only = simulate_rare_species(rare_barrier_params, t_q1, y0_q2_base, food_supply_ban)

# Polluted scenario: re-run Q1 with pollution to get degraded E(t)
polluted_web_params = FoodWebParams(
    gain_scale=ban_params.gain_scale, h_fishing=0.0, pollution=rare_polluted_params.pollution
)
_, ban_polluted_q2 = simulate_food_web(polluted_web_params, t_q1, y0_q1)
food_supply_polluted = interp1d(
    t, build_ecosystem_index(ban_polluted_q2), kind="linear", fill_value="extrapolate", bounds_error=False
)
_, polluted = simulate_rare_species(rare_polluted_params, t_q1, y0_q2_polluted, food_supply_polluted)
q2_summary = plot_q2_comparison(t2, base, polluted)
q2_summary["barrier_end"] = q2_end_values(barrier_only)
no_release_params = RareSpeciesParams(**{**rare_base_params.__dict__, "release_s": 0.0})
_, no_release = simulate_rare_species(no_release_params, t_q1, y0_q2_base, food_supply_ban)
q2_summary["no_release_end"] = q2_end_values(no_release)
q2_summary["release_lift"] = {
    "M": float(q2_summary["base_end"]["M"] - q2_summary["no_release_end"]["M"]),
    "D": float(q2_summary["base_end"]["D"] - q2_summary["no_release_end"]["D"]),
    "A": float(q2_summary["base_end"]["A"] - q2_summary["no_release_end"]["A"]),
    "S": float(q2_summary["base_end"]["A"] - q2_summary["no_release_end"]["A"]),
    "V": float(q2_summary["base_end"]["V"] - q2_summary["no_release_end"]["V"]),
}
q2_summary["anchors"] = {
    "policy_start_year": 2021,
    "gazette_year": 2022,
    "initial_counts": {
        "porpoise": 1249,
        "sturgeon": 438,
    },
    "attachment_1": {
        "facts": ["migration path", "barrier", "fish pass"],
        "mapped_parameter": "barrier",
    },
}

```

```

    "attachment_2": {
        "facts": ["food-web relations", "supplemental release"],
        "mapped_parameters": ["a_d", "a_s", "a_v", "release_s"],
    },
    "upstream_proxy": {
        "source": "Q1 ecosystem index E(t)",
        "role": "forcing input, not external observation",
    },
}
}
q2_summary["parameter_roles"] = {
    "DO_A0": "2022 gazette initial anchors for porpoise and sturgeon",
    "barrier": "attachment-1 constraint carried by the normalized corridor-loss parameter",
    "upstream_proxy": "internal food-supply proxy inherited from q1 rather than an external observation",
    "a_d_a_s_a_v": "heuristic structural priors for species interaction intensity",
    "release_s": "scenario-control parameter for artificial release rather than a fitted observation",
    "sturgeon_barrier_multiplier": "sturgeon-specific amplification of corridor blockage relative to porpoise",
}
}
q2_summary["focus_relatives"] = {
    "sturgeon_release_ratio": float(q2_summary["base_end"]["A"] / q2_summary["no_release_end"]["A"] - 1.0),
    "porpoise_barrier_ratio": float(q2_summary["barrier_end"]["D"] / q2_summary["base_end"]["D"] - 1.0),
    "sturgeon_barrier_ratio": float(q2_summary["barrier_end"]["A"] / q2_summary["base_end"]["A"] - 1.0),
    "porpoise_pollution_ratio": float(q2_summary["polluted_end"]["D"] / q2_summary["base_end"]["D"] - 1.0),
    "sturgeon_pollution_ratio": float(q2_summary["polluted_end"]["A"] / q2_summary["base_end"]["A"] - 1.0),
}
}
q2_summary["scope_note"] = "scenario output under a 2022-normalized anchor; not an independent external validation"
q2_scenarios = {
    "base": rare_base_params,
    "barrier": rare_barrier_params,
    "polluted": rare_polluted_params,
    "no_release": no_release_params,
}
}
q2_summary["upstream_proxy"] = summarize_q2_upstream_proxy(t, ban)
q2_summary["upstream_proxy"]["weight_scan"] = run_q2_proxy_weight_scan(
    t=t,
    ban_series=ban,
    polluted_q1_series=ban_polluted_q2,
    t_span=t_q1,
    y0_q2_base=y0_q2_base,
    scenario_params=q2_scenarios,
)
}
q2_summary["barrier_multiplier_scan"] = run_q2_barrier_multiplier_scan(
    t=t,
    t_span=t_q1,
    y0_q2_base=y0_q2_base,
    scenario_params=q2_scenarios,
    food_supply_ban=food_supply_ban,
    food_supply_polluted=food_supply_polluted,
)
}
q2_summary["directional_validation_2024"] = compute_q2_directional_validation(t2, base)

# Q2 sensitivity analysis: ±20% perturbation on 6 key parameters
print(["info"] Running Q2 sensitivity analysis...)
q2_summary["sensitivity"] = run_q2_sensitivity(rare_base_params, t_q1, y0_q2_base, food_supply_ban)
print(["info"] Q2 sensitivity done.)

q34_cache_ready = (
    isinstance(existing_results.get("q3"), dict)
    and isinstance(existing_results.get("q4"), dict)
    and (FIG_DIR / "fig03_q3_hastings_powell.png").exists()
    and (FIG_DIR / "fig04_q4_lyapunov_scan.png").exists()
)

```

```

if q34_cache_ready:
    print("[info] Reusing existing Q3/Q4 summaries and figures; this refresh focuses on the changed Q1/Q2
          outputs.")
    q3_summary = existing_results["q3"]
    hp_summary = existing_results["q4"]
else:
    # Question 4: chaos scan
    hp_summary = plot_hp_results()

    # Extended Q4 robustness testing
    print("[info] Running extended Q4 robustness tests (75 combinations)...")
    q4_robustness = extended_q4_robustness_test()
    hp_summary["extended_robustness"] = q4_robustness
    print(
        "[info] Q4 robustness: "
        f"{q4_robustness['overall_statistics']['passed']}/{q4_robustness['overall_statistics']['total']} "
        f"tests kept the sign-crossing bracket ({q4_robustness['overall_statistics']['pass_rate']*100:.1f}%)"
    )

    q3_summary = {
        "control_parameter": hp_summary["control_parameter"],
        "observation_window": hp_summary["observation_window"],
        "representative_behaviors": hp_summary["representative_behaviors"],
        "scope_note": "representative long-term trajectories only; finite-time MLE drift is handled separately
                      in q4",
    }

# Question 5: scenario comparison (reuses food_supply_polluted from Q2 block above)
y0_q2_inv = np.array([0.62, 1.00, 0.14, 0.10], dtype=float)
t3, rare_mixed = simulate_rare_species(
    RareSpeciesParams(barrier=0.05, pollution=0.28, release_s=0.012, a_v=0.48, e_v=0.42),
    t_q1,
    y0_q2_inv,
    food_supply_polluted,
)

scenarios = {
    "Ban only": (t2, base),
    "Ban + pollution": (t2, polluted),
    "Ban + pollution + invasion": (t3, rare_mixed),
}

q5_summary = plot_q5_scenarios(scenarios)
q5_summary["scenario_setup"] = {
    "Ban only": {
        "barrier": 0.015,
        "pollution": 0.0,
        "release_s": 0.012,
        "a_v": 0.36,
        "e_v": 0.38,
        "initial_state": [0.62, 1.00, 0.14, 0.02],
    },
    "Ban + pollution": {
        "barrier": 0.05,
        "pollution": 0.28,
        "release_s": 0.012,
        "a_v": 0.36,
        "e_v": 0.38,
        "initial_state": [0.62, 1.00, 0.14, 0.02],
    },
    "Ban + pollution + invasion": {
        "barrier": 0.05,
        "pollution": 0.28,

```

```

        "release_s": 0.012,
        "a_v": 0.48,
        "e_v": 0.42,
        "initial_state": [0.62, 1.00, 0.14, 0.10],
    },
}

results = {
    "q1": q1_summary,
    "q2": q2_summary,
    "q3": q3_summary,
    "q4": hp_summary,
    "q5": q5_summary,
}

summary_path = OUT_DIR / "summary.json"
try:
    with open(summary_path, "w", encoding="utf-8") as f:
        json.dump(results, f, ensure_ascii=False, indent=2)
except PermissionError:
    summary_path = Path.cwd() / f"summary_refresh_{datetime.now().strftime('%Y/%m/%d_%H/%M/%S')}.json"
    with open(summary_path, "w", encoding="utf-8") as f:
        json.dump(results, f, ensure_ascii=False, indent=2)
    print(f"[warn] skipped locked summary path, wrote {summary_path.name} instead")

print(json.dumps(results, ensure_ascii=False, indent=2))

if __name__ == "__main__":
    main()

```